University of Sheffield

File-Monitoring Intrusion Detection System

Sumeet Satish Punia

Supervisor: Gergely Buday

A report submitted in fulfilment of the requirements
for the degree of MSc in Cyber Security and Artificial Intelligence

in the

School of Computer Science

September 15, 2025

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Sumeet Satish Punia

Signature: Sumeet Satish Punia

Date: 15-09-2025

Abstract

Title: File-Monitoring Intrusion Detection in Standard ML: A Typed, Auditable Windows Prototype
Programme: MSc Cybersecurity & Artificial Intelligence, Department of Computer Science, The University of Sheffield

This dissertation builds a small, explainable *file-monitoring intrusion detection system* (FM-IDS) in Standard ML (SML). A thin C adapter listens to Windows file events (`ReadDirectoryChangesW`) and streams them to a typed SML pipeline. The system keeps an immutable file-integrity baseline (paths, metadata, SHA-256) and, in real time, normalises events, counts activity per folder, and scores bursts with simple statistics (z-scores) and folder-specific thresholds. A clear fusion rule turns these signals into alerts, written as compact JSON Lines (JSONL) with the evidence used (policy hit, counts, W, μ, σ, z, and, when relevant, old→new hashes). All outputs are append-only, so any alert can be replayed exactly. Platform details are isolated behind a single `MONITOR` interface, providing a clean path to a Linux `inotify` adapter without changing detection logic.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Cybersecurity risks are increasingly becoming sophisticated, covert and larger. IDS still continue to form a fundamental point of defence, as it attempts to spot any malicious activity and misuse of the system [1]. Examples of classical IDS methods are signature matching and specification rules, which find deviations relative to learned baselines, anomaly detection, which quantifies the deviations relative to the learnt baselines [2–4]. Host based IDS that measures file behavior are particularly useful given that, not only system directories are vital towards operating system integrity, but also application directories, which have been the usual target in peristence and privilege escalation attack often launched by attackers [1].

The research paper is a dissertation on a lightweight and auditable file monitoring intrusion detection system (FM-IDS) that has its functional core code in Standard ML (SML). Its implemented system is able to target windows, with a small c/ffi bridge to `ReadDirectoryChangesW` in order to receive real-time directory change events, and can be adapted to linux via inotify without modification of the SML detection (CODE) logic [5, 6]. Its core combines file integrity outlook (FIM) with per-team the statistical figures (sliding screen, z-scores) and policy verbals, releasing brief alerts of the summary its form: such as what has changed and how strange it was.

1.2 Project Scope

The scope is to design and implement a single-host FM-IDS with:

1. **Real-time ingestion (Windows).** A C wrap around `ReadDirectoryChangesW` calls and reads the normalised events to SML using FFI. [5].
2. **File Integrity Monitoring (FIM).** Governed baselines (path, size, mtime, SHA-256) detect tamper/drift via diffing and selective re-hashing [7, 8].
3. **Per-folder anomaly detection.** Sliding windows maintain rolling μ, σ; current rate x_t is scored with $z = (x - \mu)/\sigma$; optional entropy flags distributional irregularity [4].

4. **Policy and fusion.** Allow-list checks and folder-class thresholds are fused with statistical/integrity signals into a single, explainable decision [1].

5. **Evidence-rich alerts.** Line-delimited JSON includes scope, policy hit, z-score, and FIM deltas, suitable for dashboards or SIEMs.

6. **Portability path (design).** A stable SML `MONITOR` signature isolates OS specifics so a Linux `inotify` adapter can be added later without changing the detection core [6].

The prototype emphasises correctness, explainability, and small footprint on commodity endpoints; it does not require an external server or database.

1.3 Aims and Objectives

Aim: Build a robust, extensible FM-IDS with functioning SML core reporting integrity breaches and anomalous file activity in real time, and supporting every alert with preserved evidence.
Objectives:

- Develop clean adapter boundary (C/FFI $\leftrightarrow$ SML) and event normalisation format.
- Roll out FIM baseline build, immutable storage, and diffing with cryptographic hashes [8].
- Take on per-folder statistical analysis (sliding windows, rolling μ, σ, z-scores) and optional entropy [4]..
- Take on allow-list policies and threshold rules; combine with statistical and integrity signs into severity-weighted decisions [1].
- Emit standards-conformant JSON alerts and retain minimal state (μ, σ, z, window size, hashes) to replay decisions.
- Test detection quality, latency, footprint, and explainability on benign and adversarial workloads.

1.4 Research Questions and Contributions

RQ1. Can a small, functional core in SML yield a clear, testable, and auditable FM-IDS for real-time Windows monitoring? **RQ2.** Does combining FIM evidence with per-folder statistics constrain false positives without making ransomware-like bursts unobservable? **RQ3.** Is a fixed adapter interface enough for cross-OS portability (to `inotify`) without changing the detection core?

Contributions:

1. **Working Windows FM-IDS prototype** with C/FFI watcher (`winWatcher.c/.exe`) and an SML core (`alertEngine.sml`, `anomalyDetection.sml`, `policyRules.sml`, `statePersistence.sml`, `tailer.sml`, `dataLogger.sml`).
2. **Governed FIM baselines** with immutable storage and explicit update procedure; selective re-hashing to limit cost [8].
3. **Per-folder anomaly analytics** (sliding windows, rolling μ, σ, z-scores; optional entropy) and **transparent fusion** with policy thresholds and FIM results [1, 4].
4. **Explainable JSON alerts** to capture the evidence needed to reproduce each decision, making it easier for the operator to triage and audit.
5. **Portability design** via an SML `MONITOR` signature to enable the integration of a Linux `inotify` adapter with no modification needed in the SML detection modules [6].

1.5 Relationship to Programme

With its focus on explainable, defensible and auditable tooling, the project combines the secure systems engineering (host monitoring, integrity evidence, auditable decisions), and statistical anomaly analysis as part of the MSc Cybersecurity and Artificial Intelligence focus on defensible explainable tooling [1, 4]. Language-paradigm competence is shown by the use of a functional language (SML), which may be used to positively influence development based on high-assurance by providing immutability, strong typing, and composition.

1.6 Dissertation Structure

- **Chapter 2: Literature Survey** — IDS development; FIM; OS event facilities; anomaly detection; functional programming for security; research gaps.
- **Chapter 3: Requirements and Analysis** — Stakeholders, scope, FR/NFR, context and architecture, data/interfaces, key workflows, risks, and evaluation plan.
- **Chapter 4: Implementation and Technical Approach** — C/FFI Windows watcher; SML normaliser, FIM engine, anomaly module, fusion and alerting; state persistence.
- **Chapter 5: Evaluation** — Workloads, methodology, metrics (quality/ latency/ footprint/explainability), ablations, and results.
- **Chapter 6: Discussion** —Limitations (e.g., memory-only activity), portability lessons, and operator considerations, findings.
- **Chapter 7: Conclusion and Future Work** — Summary, contributions, and extensions (Linux adapter, richer features, policy learning).

Chapter 2

Literature Survey

2.1 Introduction

For more than three decades, Intrusion Detection Systems (IDSs) have continued being an integral part of practice in the information security domain. With more complicated systems, the means for behaviour monitoring have developed from signature matching to anomaly detection and hybrid methods. In this continuum, *file monitoring* is of utmost importance: not only are system directories core elements to the stability of the operating system, but also one common target of attackers to achieve persistence or privilege escalation [1].

This paper studies a *file-monitoring IDS* (FM-IDS) written in *SML* for Windows, but with a design that is transportable to Linux. The review includes: (i) the history of IDS and FIM; (ii) the support from operating systems for detecting real-time file events (`ReadDirectoryChangesW` in Windows, `inotify` on Linux); (iii) anomaly detection techniques ('traditional' statistical ones and threshold-based); and (iv) why functional programming—and particularly SML—can be of use to system security. Along the way, we highlight the originality of our contribution and how prior work is related to the practical modules we have developed in our project: Windows API bridge through C/FFI, recursive directory monitoring, FIM baselines per folder sliding windows z-score and threshold policies JSONL-based alerting.
JSONL. Here, the line-delimited variant of JSON (JSON Lines) is named due to its use as a log format; every record in the log is its own line (UTF-8) of JSON (duplicating the initial use of JSON) and therefore the streams are append-only, can be processed by a stream program, and can be diffed to compute the difference between two streams [JSONLines] [9].

2.2 Intrusion Detection Systems

2.2.1 Historical context

Denning's model introduced anomaly detection based on a statistical software profile of what is normal namely baselines and deviations to the core of IDS [10]. Signature-based solutions such as Snort could achieve high accuracy when they were used to detect known threats, but

had difficulty in case of zero-day exploits [2]. Generalizing detection, anomaly-based systems had traditionally been plagued with more false positives [3]. Combining these two paradigms led to the hybrid IDS, which is more robust and widely used in practice [11].

2.2.2 Host-based IDS (HIDS) and FIM

Host-based IDS (HIDS) Checking on system-centric artifact such as files, logs and also process. One of the HIDS approaches is File Integrity Monitoring (FIM), where cryptographic baselines are compared with current states to detect integrity violations [1]. Earlier systems such as Tripwire solidified this approach with a commitment-based hash verification [7]. OSSEC and Wazuh have later combined the FIM with system logs analysis based on rules, active responses, and dashboards [12–14]. Standards such as PCI-DSS explicitly demand file integrity controls [15].

2.3 File Integrity Monitoring (FIM)

2.3.1 Principles

FIM creates a secure *baseline snapshot* of critical paths, and that anything inconsistent with such baseline would be an evidence of tampering. For each file, the baseline typically preserves a canonicalised path and size, creation/modification/access timestamps and a cryptographic hash of the content (e.g., SHA-256) such that any change at bit level directly changes logged signature [8]. At runtime, the system compares the baseline with the current state to identify creations, deletions, modifications and renames. This "state-and-diff" approach forms the foundation for classic tools such as Tripwire, and continues to be recommended in host-based IDS and compliance guidelines [1, 7]. For our astronomy application, the aim is to monitor Windows system services and user directories — paths such as `C:\Windows\System32` and `C:\Users`, with a conservative default posture: sensitive folders are inspected more closely than user space.

2.3.2 Baseline construction and governance

A strong assertion does not sound like "baseline is a one-off scan but it's also a *control-led, versioned, auditable artefact*". For this dissertation, by *governance*, is referred to the fact that there is indeed an owner for the baseline, a well-defined and documented change process(who did what when why), provenance(stored history so decisions can stand independent verification) [1]. This is consistent with operational practice as implemented in OSSEC/Wazuh, where policy, inventory and FIM outputs are treated as distinct, auditable documents [13, 14].

Practical steps. We (i) scope up front (watched roots, high-value subtrees, and explicit exclusions for caches/ephemera); (ii) walk safely with symlink/reparse-point awareness; (iii) record canonical path, size, modify time, and strong content hash (e.g., SHA-256) to provide tamper evidence; (iv) store the snapshot immutably as append-only lines so attacker changes cannot be silently "learned"; (v) update only through change control (propose → review →

approve), typically after a signed or otherwise authorised update; and (vi) keep prior versions with provenance metadata (approver, timestamp, rationale) and a content hash for future verification. These governance steps prevent silent drift and ensure any alert based on the baseline can be recomputed and audited from first principles [1, 13, 14].

2.3.3 Runtime monitoring and change semantics

FIM can run *event-driven* (take file-system notifications as they happen) or *scheduled* (periodic re-scan). Our design favors event-driven checks for freshness, complemented by targeted re-verification (rehash/re-stat) to guarantee integrity when a burst of changes is noticed. This process rename/move operations carefully: in the absence of stable identifiers, a rename will look like a delete+create pair; the pipeline therefore normalises sequences of events within a window to avoid duplicate alerts. Partial writes and temporary temp files are combatted with short back-off waits and re-stat before hashing, reducing false positives from transient editor saves. Finally, FIM alerts provide feed to the anomaly layer as escalation signals (e.g., an integrity hit escalates the severity of a rate-based anomaly occurring simultaneously), producing alerts that combine *what changed* with *how unusual* the activity was.

Windows event semantics and pitfalls. Under load on Windows, directory notifications can pile up, and rename operations show up as double old/new events; lost pairs may surface as delete+create. Buffers for `ReadDirectoryChangesW` must be dimensioned carefully so as not to overflow, and long rename chains may become reordered under stress. The pipeline handles these edge cases by (i) batch and normalising events over a brief window, (ii) re-stating suspicious paths before hashing, and (iii) conservatively handling unmatched rename pairs until checked by follow-up events.

2.3.4 Scope, Exclusions, and Noise Control

Choosing appropriate scope is fundamental to signal quality. Project exclude high-churn locations (browser caches, tmp dirs) unless for a case study and would favor system binaries, config stores, and app dirs. Exclusions are based on policy and logged, so behavior is reproducible. Under NTFS, it is observed that attackers keep artefacts in less-nobly-discernible spots (alternate data streams); the foundation policy may be extended to enumerate such features if needed. To avoid alert storms, we consolidate folder- and time-window-related file events, reduce duplicates after diff engine, and provide a lone incident with summery evidence fields (`path`, `baselineHash`→`newHash`, `timestamp`, `policyHit`) [1].

2.3.5 Performance, Storage, and Tamper Evidence

It is expensive to hash large files: therefore, short-circuit rehashing when size and mod-time have not changed, and defer content hashing to very large files until a metadata change is confirmed. Baselines are stored in a stable, machine-readable format (e.g., line-delimited JSON) to enable easy diffing, transport, and signing. The use of collision-resistant digests

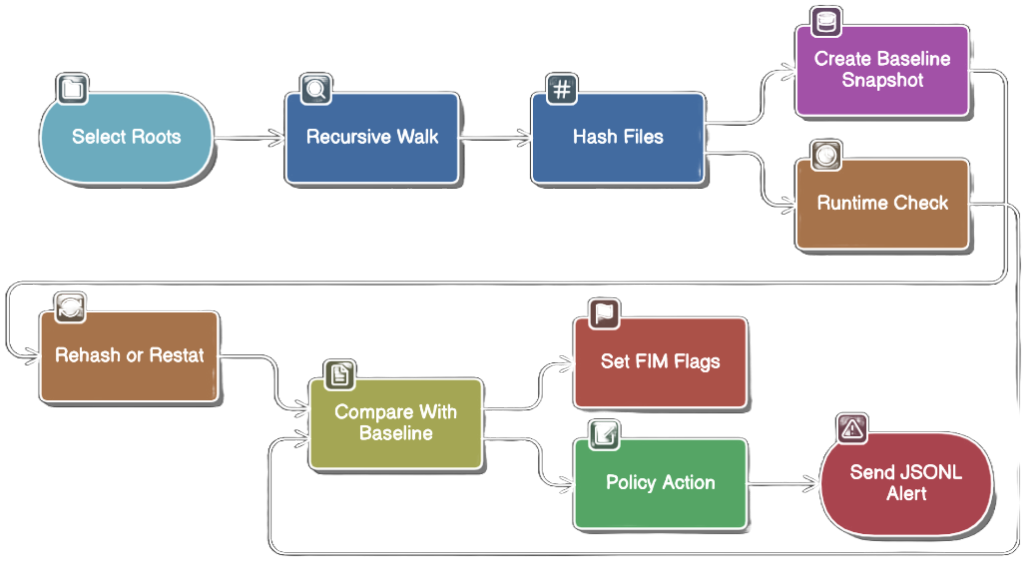

Figure 2.1: FIM pipeline: immutable baseline, runtime comparison, policy-led alerting to line-delimited JSON (JSONL).

(e.g., SHA-256) is strong evidence of content integrity [8]; the use of hashes in conjunction with metadata speeds common-case checks at the expense of preserving cryptographic proof when content changes. These choices honor performance trade-offs found in operational FIM tools [13, 14].

2.3.6 FIM workflow diagram

2.3.7 Industry tools, compliance, and trade

OSSEC and Wazuh integrate FIM with log correlation and active response, and offer mature rulesets and dashboards [13, 14]. Their scale is beneficial for use in enterprise environments, however operational footprint and config complexity are potential, and imperative codebases are harder to reason about at scale [12]. This prototype's approach is lightweight, such that the FIM core is small, easy to audit, and composable, yet generates standards-friendly evidence consumable by downstream systems. This aligns with common audit and compliance needs (e.g., PCI DSS control families) that must have demonstrable change governance and file integrity controls [1, 15].

Limits of FIM. As well-documented in practice, FIM does not monitor memory-only attacks, payloads encrypted that decrypt at run-time, or tampering that takes place completely between scans without later reads. Integrity checks are most effective when coupled with real-time events and behavioural context—adopted as is herein, so that *what changed* is complemented

by *how strange* a change-pattern was [1,8].

Relative positioning . In order to position this work against current practice, Table 2.1 juxtaposes canonical FIM/host-based methods with our adopted lightweight, functional approach.

	Tripwire	**OSSEC/Wazuh**	**This work (SML)**
Core focus	Baseline/diff (hash)	FIM + logs + active resp.	FIM + per-folder anomaly
Agent/server	None / standalone	Agent + mgr + ruleset	Agent-only, no server
Event ingestion	Periodic scan	Realtime + scan	Realtime (Windows), scan assist
Policy	Signatures/rules	Extensive rules + decoders	Typed SML policies (small, auditable)
Footprint/complexity	Low/medium	Medium/high	Low
Explainability	Hash diffs	Alerts + dashboards	JSONL evidence (hash, z-score, counts)
Portability path	Cross-OS scans	Cross-OS agents	Adapter via SML signature

Table 2.1: Comparative view: classic and integrated tools versus this lightweight SML approach.

2.4 Operating System Support for File Events

2.4.1 Windows API via C/FFI

Windows offers real-time directory events via the function `ReadDirectoryChangesW` [5].This bundle such an API in a brief C program (compiled for ucrt64) and provides a structured event stream via SML's Foreign Function Interface (FFI). This keeps performance in check while keeping SML code purely functional where possible.

```
1  void monitorDir(const char* folder) {
2      HANDLE hDir = CreateFileA(folder, FILE_LIST_DIRECTORY,
3          FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
4          NULL, OPEN_EXISTING,
5          FILE_FLAG_BACKUP_SEMANTICS | FILE_FLAG_OVERLAPPED, NULL);
6
7      char buffer[8192];
8      DWORD bytesReturned;
9      while (ReadDirectoryChangesW(hDir, &buffer, sizeof(buffer), TRUE,
10             FILE_NOTIFY_CHANGE_FILE_NAME |
11             FILE_NOTIFY_CHANGE_DIR_NAME  |
12             FILE_NOTIFY_CHANGE_ATTRIBUTES|
13             FILE_NOTIFY_CHANGE_SIZE      |
14             FILE_NOTIFY_CHANGE_LAST_WRITE,
```

```
            &bytesReturned, NULL, NULL)) {
        /* marshal events -> ring buffer -> FFI -> SML */
    }
}
```

Listing 2.1: C wrapper sketch for `ReadDirectoryChangesW` (colours, line numbers, no box).

2.4.2 Linux counterpart

Linux provides the `inotify` service for real-time alerts [6]. When contrasted against Windows, `inotify` is defined by watch granularity and overflow behavior that differ; operations are seen as several alerts in certain cases. We bundle such details in a single SML `MONITOR` signature (beginning / subscribe / terminate), thus keeping the detection core (baselining, stats, fusion) invariant and allowing for cross-OS experiments [6].

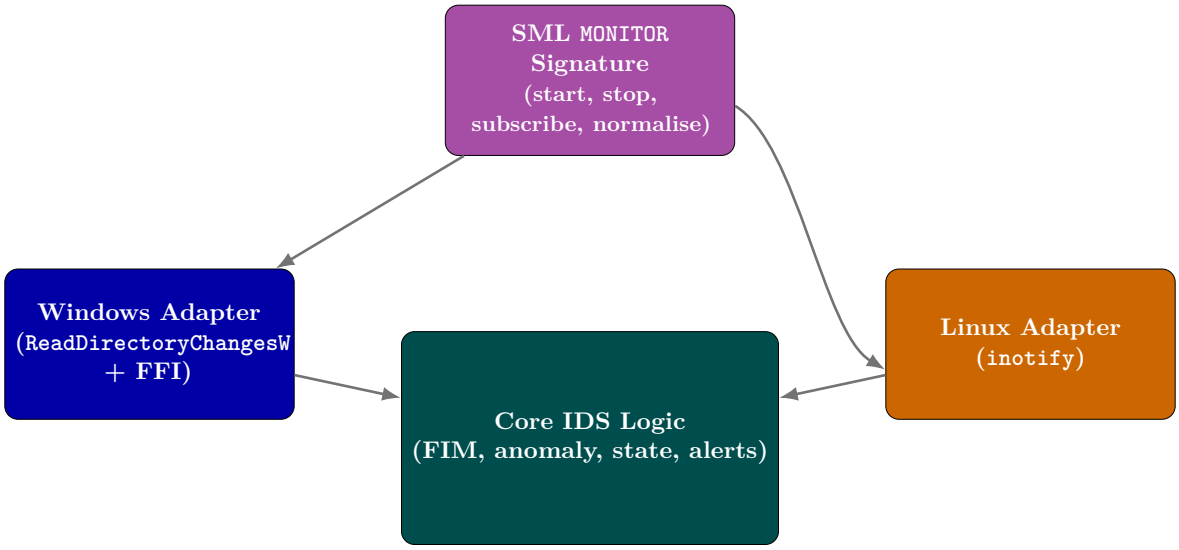

Figure 2.2: Cross-platform abstraction: platform adapters satisfy the same SML signature; core logic is shared.

2.5 Anomaly Detection for File Activity

2.5.1 Statistical methods

We measure the level of abnormality of current activity relative to each folder's most recent baseline. Define x_t, an *event rate* (e.g., `create`/`modify`/`delete`/`rename` over a minute), in a fixed window. For a watched folder, we maintain rolling statistics over the last W windows and compute the current window relative to a z-score:

$$z_t = \frac{x_t - \mu_t}{\sigma_t}.$$

Large positive values of z_t signify bursts, such as ransomware-like sprees, whereas values that are near zero indicate routine behavior [4]. To ensure robustness in practical applications, we implement several measures: (i) a warm-up period is utilized prior to scoring, (ii) very small values of σ_t are clamped with a small ε to prevent division spikes, and (iii) light winsorisation may be optionally applied to prevent a single extreme window from distorting future baselines. Utilizing per-folder baselines prevents the penalization of legitimately busy locations and allows for tighter control on sensitive paths, such as `C:\Windows\System32`. To promote reproducibility across system restarts, rolling statistics and window parameters are stored alongside alerts and subsequently reloaded upon startup, thereby ensuring that z-scores remain stable over time.

2.5.2 Threshold fusion and alert logic

In threshold fusion and alarm logic, threshold rules are still efficient in discovering high-volume assaults and uncomplicated patterns of misuse [1]. We thus sum two orthogonal signals:

$$\text{(i) a } \textit{statistical} \text{ trigger when } z_t \geq z_{\text{thr}}$$
$$\text{(ii) an } \textit{absolute-rate} \text{ trigger when } x_t \geq X_{\text{thr}}(\text{folderClass}).$$

Besides rate-based evidence, we calculate a simple distributional feature—the Shannon entropy of current action counts per window—to mark abnormally "spread out" action that is not uncommon in automated tampering or bursty script attacks. The results of file integrity (baseline hash vs current hash) are used as multipliers: a tamper flag raises severity; absent tampering, the same anomaly is written out at a lower level to facilitate context. We continue to keep $(\mu_t, \sigma_t, x_t, z_t, W)$ with each alert so that decisions are always auditable and explainable [1, 4].

Entropy supplements rate-based indications by registering distributional irregularity over actions or paths; bursts that contact a large number of files with similar probabilities expand entropy when raw counts are variable. Previous surveys of anomalies suggest joint use of rate and distributional indicators in order to enhance sensitivity to automation behaviours [4].

Severity	**Statistical / Absolute**	**Notes**
High	$z_t \geq 3.5$ or $x_t \geq X_{\text{high}}$	Escalate one level if FIM tamper is true.
Medium	$z_t \geq 2.5$ or $x_t \geq X_{\text{med}}$	Folder-class specific limits.
Info	$z_t \geq 2.0$	Log only; contributes to trends.

Table 2.2: Fusion policy combining z-scores with absolute-rate thresholds.

2.5.3 Alert engine (project implementation)

The alert engine determines how interesting the current folder and path are through use of (a) the z-score of rate bursts, (b) the Shannon entropy of current action counts, and (c) allow-list policy rules. The output level is then built into a concise, machine-readable alert that notifies the visualization layer and downstream tools.

```
(* alertEngine.sml*)

structure AlertEngine =
struct
  open AnomalyDetection
  open MLAnomaly
  open PolicyRules

  datatype alertLevel = INFO | WARN | CRITICAL

  fun levelToString l =
    case l of
        INFO => "INFO"
      | WARN => "WARN"
      | CRITICAL => "CRITICAL"

  (* Main scoring logic based on z-score and entropy *)
  fun evaluate (folder:string, path:string) : alertLevel =
    let
      val z = zScore folder
      val ent = shannonEntropy (countsForStats folder)
      val allowed = isAllowed path
    in
      if not allowed andalso Real.abs z > 3.0 andalso ent > 2.0 then CRITICAL
      else if Real.abs z > 2.0 orelse ent > 3.0 then WARN
      else INFO
    end

  (* Format full alert message *)
  fun makeAlert (timestamp:string, action:string, path:string, folder:string) =
    let
      val level = evaluate (folder, path)
      val message =
        "[ALERT][" ^ levelToString level ^ "] " ^
        timestamp ^ " | " ^ action ^ " | " ^ path ^ " | Folder: " ^ folder
    in
      (level, message)
    end
end
```

Listing 2.2: Project alert engine: score fusion (z-score, entropy, policy) and formatted alert message.

2.5.4 Chronological workflow

The end-to-end flow is: (1) normalised events arrive from the OS watcher; (2) events are bucketed to fixed-sized windows by folder; (3) we sum counts together to form x_t; (4) μ_t and

σ_t are updated by rolling stats; (5) we compute z_t; (6) absolute thresholds are checked for the folder class; (7) z-score, entropy, allow-list and FIM tamper are folded into a combined decision with a cool-down; (8) we output both the alert and state (profiles, μ, σ, window size) for explainability and trend charts.

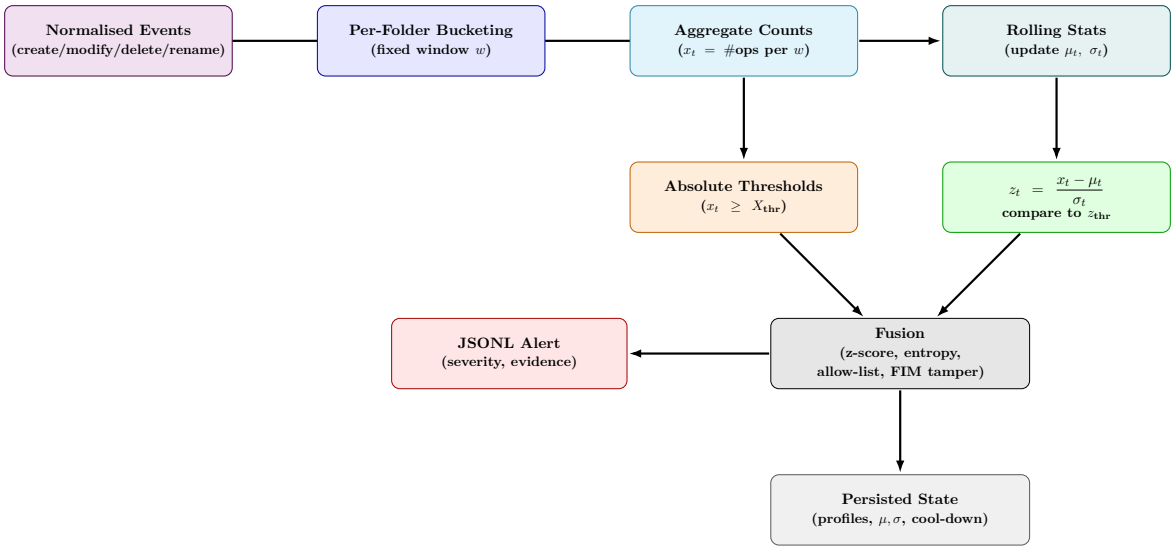

Figure 2.3: Anomaly workflow: event counting → rolling statistics → z-score and absolute checks → fusion (entropy, allow-list, FIM) → JSONL alert and persisted state.

2.6 Alerting and Visualisation

Operator effectiveness is contingent upon *interpretability*. Alerts are emitted in the form of *line-delimited JSON (JSONL)* and include fields such as `timestamp`, `folder`, `eventType`, `policyHit`, `zscore`, `baselineHash`, `newHash`, and `decision`. This format integrates seamlessly with dashboards and SIEM platforms [1].

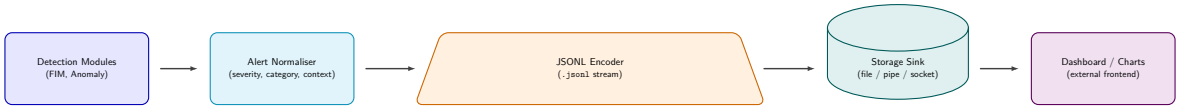

Figure 2.4: Alerts are encoded as line-delimited JSON (JSONL) for downstream analysis and charting.

2.6.1 Alert schema and severity

The writing stresses compact, well-structured alerts with sufficient context to facilitate fast triage and automation handling [1, 11]. In this project, every alert contains evidential information describing *why* it triggered (e.g., threshold reached vs. statistical anomaly), as

well as pre/post file metadata in order to facilitate verification. A light-weight severity scheme (`info`/`warning`/`high`) is calculated from the folder class (e.g., `C:\Windows\System32` vs. user space), how far it deviated from normal behavior (z-score), and whether its contents have been modified (hash changed). The schema is consistent between integrity and anomaly detections such that downstream tools may take a uniform approach to handling alerts.

Field	Purpose
`timestamp`	Event time (UTC) for correlation and timelines.
`folder`, `path`	Scope and impacted object for drill-down.
`eventType`	`create`/`modify`/`delete`/`rename`.
`policyHit`	Which rule fired (e.g., `fim_tamper`, `rate_burst`).
`zscore`	Magnitude of deviation for behaviour-based alerts.
`baselineHash` → `newHash`	Integrity diff evidence (FIM).
`severity`, `confidence`	Operator-facing priority and model confidence.
`evidence[]`	Minimal facts used in the decision (counts, window size, folder class).

Table 2.3: Minimal, consistent alert schema to support triage and automation.

2.6.2 Aggregation and correlation

In order to reduce alert storms and lighten analysts' workloads, alerts are time windowed and foldered, and de-duplicated if several rules fire against a same underlying burst [1]. 1–5 minute rolling correlation windows allow aggragation of correlated file events into a singular incident with a coherent context, yet retaining raw evidence for audit purposes. This is in keeping with recommended practice reviews in favour of normalisation, correlation, and prioritization, before presentation to operators or Security Information and Event Management systems (SIEMs) [11].

2.6.3 Visual encodings for operator workflows

Good visualization emphasizes *change over time* and *where* it occurred. Standard, low-friction displays are: (i) per-folder *rate timelines* (events/min) with banded thresholds; (ii) *heatmaps* of folders (rows) versus time (columns) to expose bursts and diurnal behavior; (iii) *Pareto bars* for "top-N noisy paths" to inform remediation; and (iv) *z-score histograms* to separate normal from abnormal behavior [1, 4]. Every alert is linked back to its evidentiary basis (hashes, counts, policy hits), so drill-down from chart to underlying record is available for fast.

2.7 Functional Programming and Security

2.7.1 Language fit

Functional programming shares characteristics of immutability, higher-order composition, strong typing, and pattern matching—which promote clarity and verifiability in security-critical

code [16]. These characteristics align well with FIM, or immutable baselines, as well as anomaly pipelines, or composable transformations, so that detection logic may itself beSpecified as small, testable functions that have explicit data flow rather than side effects..

2.7.2 Academic precedents

Functional methods have been applied in protocol verification [17] and type-safety work [18]. Nevertheless, functional programming's practical file-monitoring IDS are not well-explored, providing a gap to assess SML for practical host monitoring where correctness, auditability, and maintainability are no less vital.

2.7.3 Typed event pipelines and policy safety (application to this work)

An example from this work. Here, raw OS change notifications (via C/FFI interface to `ReadDirectoryChangesW`) are normalized into typed SML events beforehand, during any detection step [5]. Separating *raw* from *normalized* events at the type level and using exhaustive pattern matching, detectors accept only verified inputs, and policy code (folder classes, thresholds, severities) remains entire and deterministic—with fewer misconfigurations and a less challenging pipeline to reason about and verify.

2.7.4 Immutable baselines and explainable alerts

FIM snapshots are treated as immutable values; changes are computed by pure "diff" functions comparing current hashes and metadata to saved baselines [8]. This architecture produces reproducible conclusions and compact, machine-readable notices (JSONL) that outline *what changed*, *how anomalous* (e.g., z-scores), and *why it was flagged*—thus facilitating operator triage and downstream tooling in line with host-based IDS best practices..

2.8 Research Gaps and Motivation

Although host-based intrusion detection and file-integrity monitoring are well-established areas, there are three practical gaps. First, functional programming is scarcely considered in applied IDS deployments, and most of the systems are based upon imperative approaches, which scale poorly in terms of reasoning [16]. Second, lightweight, auditable prototypes are scarce, for the ecosystem based itself upon heavyweight agent-manager sets that increase operational complexity, yielding few prototypes that, however, provide real-time signals [12]. Third, cross-platform abstractions are few; Windows and Linux have diverging file-event semantics (`ReadDirectoryChangesW` vs. `inotify`), although reducing portability and comparative analysis, as well as, incidentingly, leaving underexploited host-side behavioral signals and explainable, machine-readable alerts [1, 4, 6].

This dissertation endeavours to explore whether a functional core—"write once, run everywhere"—crafted in Standard ML may augment clarity, modularity, and verifiability in an operational host Intrusion Detection System (IDS). A lightweight C/Foreign Function

Interface (FFI) bridge reads Windows file events from `ReadDirectoryChangesW`, whereas the SML layer stores immutable baselines and facilitates composable pipelines for normalization, integrity checks, and lightweight behavioral analytics, such as sliding windows, z-scores, and policy thresholds [4,5,8]. Design aims at generating concise, evidence-dense alerts interpretable easily by operators and consumable readily by downstream tools [1].

The ultimate motivation is practical and pedagogical; it is to provide a lightweight, reproducible prototype usable independently of server infrastructure yet extensible and interchangeable across variable operating platforms. An robust SML `MONITOR` interface is used to decouple platform details such that a Linux `inotify` adapter could be incorporated without changes to detection logic, supporting cross-operating system research as well as eventual enhancement [6]. The work, in this way, offers a streamlined alternative to bloated technology stacks as well as providing a foundation for continued academic research and use in the teaching lab [1, 12].

2.9 Summary

This chapter traced the evolution of intrusion detection from early statistical models and signature-based systems to hybrid host-centric models that position *file integrity monitoring* at the center of endpoint defense [1, 10, 11]. We codified FIM as a controlled baseline–diff process logging canonical paths, metadata and crypto digests, and we discussed operational practice for scope definition, exclusions, and immutable storage. Runtime consequences were made tangible: Windows event semantics via `ReadDirectoryChangesW` (coalescing, rename pairing, buffer size) and mitigation thereof, with portability to Linux `inotify` via a stable SML `MONITOR` interface [5, 6]. We also identified limits of FIM (e.g., memory-only activity) and pushed pairing integrity evidence by behavioral context for improved detection coverage.

Drawn from that effort, we considered anomaly detection targeted at file behavior: per-directory sliding windows, rolling statistics and z-scores to quantify bursts in comparison with local baselines, with absolute rate rules for high-rate behaviors [1, 4]. We included rates with a simple distributional indicator (Shannon entropy) and fused signals with FIM results into severity-ranked decisions. To support triage and automation, we standardized a compact JSONL alert schema (timestamp, scope, policy hit, z-score, hash diffs, evidence) and specified visual encodings that emphasize change over time and "top-N" noisy paths, following best-practice guidelines [1].

Finally, we situated the work in context of available tools (Tripwire, OSSEC/Wazuh) and established the rationale of a lightweight, auditable proof-of-concept with a functional core. Functional programming concepts—immutability, strong types, and total pattern matching—are inherent in the immutable baselines and composition anomaly pipeline of FIM, which improve clarity, testability, and policy safety [16–18]. We simply apply these from the literature directly to Chapter 3: we instantiate them into firm requirements (baseline management, event normalisation, statistical/threshold fusion, explainable alerts, cross-platform adapters) and corresponding design decisions for the SML implementation.

Chapter 3

Requirements and Analysis

3.1 Purpose and Scope

This chapter translates findings in the literature into actual specifications for a light, auditable *file-monitoring intrusion detection system* (FM-IDS) deployed in Standard ML on Windows with clean upgrade to Linux. The scope prioritizes correctness, explainability, and low run-time overhead: immutable integrity baselines and per-folder behavioural analytics generate tight, evidence-rich alarms that can be consumed directly by operators and downstream tools [1, 4]. Where mobility is a concern, we encapsulate platform information behind a single SML adapter interface so that the detection core can be kept invariant across Windows `ReadDirectoryChangesW` and Linux `inotify` [5,6]. Quality attributes (performance, reliability, testability, auditability) are first-class and used to trade-off decisions [19].

3.2 Stakeholders and User Stories

Primary stakeholders: (i) an operator (security/IT) handling alerts; (ii) a developer / researcher adding policies and modules; (iii) an assessor verifying compliance evidence (e.g., FIM controls).
Representative user stories:

- **U1 (operator):** "I want alerts with low noise but sufficient context (what changed, how anomalous, where) so I can choose quickly."
- **U2 (developer):** "I want modular, typed interfaces (SML) so I can swap out the Windows back-end with a Linux adapter without changing detection logic."
- **U3 (assessor):** "I want machine readable proof (baselines, diffs, hashes) to demonstrate file integrity controls to auditors."

3.3 Assumptions and Constraints

- **OS & APIs:** Windows 10/11 with `ReadDirectoryChangesW`; Linux adapter with `inotify` is on the plans [5, 6]
- **Privileges:** No kernel modules or admin-only drivers; userspace C/FFI bridge + SML core.
- **Footprint:** Small memory/CPU; commodity endpoints okay (no server reliance).
- **Security posture:** Baselines are set; updates must be explicitly approved.
- **Paths & encoding:** Unicode (UTF-16) and long paths are normalized by the adapter; SML receives canonical UTF-8 with normalized separators.
- **Data:** State and alerts are line-delimited JSON for easy ingestion and audit; retention / rotation is configurable as well.

Out of scope

This is host-side file activity-centric work. It does not provide kernel driver support, network intrusion detection, malware classification, or response actions EDR-style (quarantine/rollback). Machine learning models beyond simple statistical scoring (rolling μ, σ, z) are intentionally out of scope in order to ensure explainability and a small operational footprint.

3.4 Functional Requirements (FR)

Table 3.1 recapitulates the concrete capabilities the system must provide. At a high level, FR-1–FR-3 cover event ingestion and integrity baselining; FR-4–FR-5 enumerate per-folder behavioural analytics and statistical and integrity signal fusion; FR-6–FR-7 spell out evidence-rich alerting and persisted state for explainability; and FR-8–FR-10 enumerate operator configurability, cross-platform portability, and resilience to bursts via aggregation/deduplication. Together, these ensure alerts are timely and auditable, and the design is compact, testable, and portable.

Requirements Traceability (FR → Modules → Tests)

Traceability links each functional requirement in Table 3.1 to the implementing modules and to the objective tests proving them. This gives us three guarantees: (i) no requirement is missed in the implementation, (ii) there is no code without a justification (*no orphan code*), and (iii) there is a clear acceptance path for each requirement. The tracing also enables impact analysis at speed: if an FR is changed, the affected modules and tests are immediately obvious, reducing regression risk. **See Table 3.2** for the full FR→modules→tests/evidence tracing used during development and evaluation.

ID	Requirement
FR-1	Ingest file events in real time via a C/FFI bridge to `ReadDirectoryChangesW`.
FR-2	Build immutable FIM baselines (path, size, mtime, SHA-256) and store durably [8].
FR-3	Detect diffs against baseline (create/delete/modify/rename) and classify as *tamper* or *drift*.
FR-4	Maintain per-folder sliding windows; compute rolling μ, σ and z-scores [4].
FR-5	Apply folder-class thresholds; fuse with z-score and integrity outcomes into a single decision [1].
FR-6	Emit JSONL alerts with `timestamp`, `folder`, `path`, `eventType`, `policyHit`, `zscore`, `baselineHash`→`newHash`, `severity`, and `evidence[]` (counts, window W, folder class).
FR-7	Persist per-folder statistics and last decisions for explainability and review.
FR-8	Provide configuration for folder classes, exclusions, thresholds, and cool-downs.
FR-9	Expose a stable SML `MONITOR` signature so a Linux adapter can be added without changing core logic [6].
FR-10	Aggregate/deduplicate alerts within a time window to prevent storms [11].

Table 3.1: Functional requirements derived from literature and project goals.

3.5 Non-Functional Requirements (NFR)

Non-functional qualities shape the architecture just as features do. We express them here as *measurable goals* so that they guide design compromises and can be empirically confirmed. The priorities are operator experience (low latency and noise), operational safety (no silent loss, baselines under control), and evolvability (clean interfaces and testable, pure logic). Portability is managed by keeping OS details behind a single adapter interface; explainability requires that every alert be reproducible from saved evidence.

Verification and acceptance. We shall (i) report median/95th percentile of latency from receipt of an OS event to sending an alert for scripted workloads and record it; (ii) profile CPU/memory observing `C:\Windows\System32` and user directories; (iii) develop burst and rename-storm scenarios for testing bounded loss and re-stat logic [5]; (iv) substitute the Windows adapter with a Linux `inotify` implementation to ensure that the core SML modules remain unchanged [6]; (v) run unit tests on pure functions (normalise, diff, score) and ensure that a sample of alerts can be recreated in exact form from persisted state and configuration [1]. These checks constitute the acceptance criteria and serve as anchoring points for tuning decisions when deployed.

3.6 System Context and Architecture

3.6.1 Context and system boundary

The FM-IDS is a *single-host* agent between the operating system's file-event source and two consumers: (i) the human operator (or a dashboard/SIEM) who will be notified, and

FR	Primary modules / key functions	Planned tests / evidence
FR-1	`winWatcher.c` (`wmain`, RDCW loop), `winWatcher.exe`; `monitorLauncher.sml` (`spawnWatcher`); `osDetect.sml`; `ffiWatcher.sml` (FFI alt); `tailer.sml` (ingest pipeline).	Inject create/modify/rename bursts; verify lines in per-folder `events_*.jsonl`; measure ingest→alert timestamps.
FR-2	`FIMSnapshot.sml` (`snapshot`, `printSnapshot`); `hashing.sml` (`sha256_of_file`); optional `hash_ffi.c`.	Build baseline in a test root; verify presence of (path, hash) pairs; re-run after benign edits to ensure stable entries.
FR-3	`FIMSnapshot.sml` + comparison in processing path (hook from `tailer.sml`/`alertEngine.sml`); `dataLogger.sml` for evidence.	Modify a baseline file; expect alert with hash delta evidence. (Ensure baseline-vs-current comparison occurs before alert emission.)
FR-4	`anomalyDetection.sml` (`bumpCount`, `countsForStats`, `zScore`); `statePersistence.sml` (save/load).	Feed synthetic windows; check rolling μ, σ and z changes; reload state and confirm continuity.
FR-5	`policyRules.sml` (`isAllowed`); `mlAnomaly.sml` (`shannonEntropy`); `alertEngine.sml` (`evaluate`: OR fusion of $\|z\|$, entropy, allow-list).	Adversarial bursts in sensitive vs. noisy folders; verify severity mapping (INFO/WARN/CRITICAL) and escalation.
FR-6	`visualExport.sml` (`withAppend`, JSON lines); `dataLogger.sml`; `tailer.sml` (format lines).	Validate JSONL schema: `timestamp, folder, path, eventType, policyHit, zscore, severity, evidence[]`.
FR-7	`statePersistence.sml` (persist anomaly state); `anomaly_state.txt`.	Restart test: confirm decisions reproduce from stored μ, σ and window size.
FR-8	`policyRules.sml` (whitelist); small config in `tailer.sml`/`visualExport.sml`.	Change folder classes/thresholds; confirm runtime follows config and logs config errors cleanly.
FR-9	`osDetect.sml`; adapter switch in `monitorLauncher.sml`; placeholder `linuxWatcher.so`.	Swap adapter (Windows→Linux path); confirm no changes in detection modules; run same functional tests.
FR-10	`tailer.sml` (deduplicate within window); `alertEngine.sml` (cool-down in evaluate/emit flow).	Replay a bursty script; verify one incident rather than a storm; show grouped evidence.

Table 3.2: Functional requirements traced to implementation modules and tests/evidence that verify each.

Attribute	Requirement
Performance	End-to-end alert latency under typical load $\leq 2\,\text{s}$; sustained processing of ≥ 500 events/min on commodity hardware.
Footprint	Runtime memory $< 200\,\text{MB}$; CPU $< 10\%$ at idle baselines.
Portability	Core SML logic unchanged when swapping Windows adapter for Linux `inotify`.
Reliability	No silent drop of events under nominal load; bounded loss under buffer overflow handled by re-stat [5].
Security	Baselines are append-only; updates require explicit action; JSON alerts contain sufficient evidence for audit.
Testability	Pure functions for normalisation, scoring, and diffing are unit-testable; FFI isolated behind `MONITOR` signature.
Explainability	Each alert is reproducible from stored state (μ, σ, z, hashes) and policy settings [1].

Table 3.3: Non-functional requirements and quality attributes.

(ii) the audit trail where baselines, diffs, and the minimal state to reproduce decisions are maintained. It has no database or manager server need; all computation is local with small, append-only artefacts for future inspection [1]. The operating system is the upstream producer of events—`ReadDirectoryChangesW` on Windows today and `inotify` on Linux in the portability strand—while JSONL notifications and line-delimited state files are the downstream output [5, 6]. This tight boundary makes deployment simple (copy the agent, create folders, go) and confines operational risk.

3.6.2 External interfaces

Upstream, the agent subscribes to directory notifications with a minimal C bridge cross-compiled for target runtime and offers a strict, typed event stream to the SML core through the foreign function interface. Normalisation during this step prevents raw, platform-specific structures from seeping into the rest of the system and ensures that all downstream components see a homogenous tuple: `{timestamp, folder, path, eventType}` with optional `size, mtime` [5]. Downstream notifications are sent as line-delimited JSON with permanent field names so dashboards, scripts, or SIEMs can consume them without schema drift. Settings (folder classes, exclusions, thresholds, cool-downs) are provided as a small text file validated at startup; invalid settings fail fast and are returned as human-readable diagnostics.

3.6.3 Internal decomposition

Internally the design separated *volatility* (platform APIs and I/O) from *stability* (detection logic). A lightweigh **Event Adapter** in C receives OS notifications and sends normalised events to SML via FFI. The SML's foundation is a **Normaliser** guaranteeing well-formedness and canonical paths, and then it splits into two detection paths: a **FIM Engine** cross-matching available file metadata and hashes against an immutable reference to detect *tamper/drift* [8],

and an **Anomaly Module** that maintains per-folder sliding windows, rolling μ, σ, and z-scores to quantify outlier bursts [4]. There is a very small **Fusion Layer** that merges integrity findings with statistical/threshold signals into one decision and severity, the **Alert Emitter** serializing to JSONL and saving the evidence (window size, μ, σ, z, baseline and current hashes) needed to reproduce the decision [1].

3.6.4 Data flow and control

The data flow is explicitly linear: events → normalisation → detection → decision → alert. Control matters are handled locally at each step. The adapter accumulates short bursts and is robust to coalesced notifications; before expensive hashing, the pipeline re-stats paths to bypass transient editor saves or partial writes [5]. The anomaly path buckets events in fixed windows by folder so active areas take larger baselines while sensitive directories have smaller bands; thresholds are folder class-configured, and a short cool-down prevents alert storms. Fusion uses an explicit rule (logical OR with severity mapping), favoring concise explanation rather than dark scoring; integrity hits emphasize otherwise marginal anomalies. All artefacts are append-only for supporting post-mortem analysis. See Section 3.8 for end-to-end workflow.

3.6.5 Context view

The FM-IDS acts as a one-host agent between the OS file-event source and two consumers: an operator/dashboard and an audit sink. It takes in NTFS/`inotify` notifications and outputs JSONL alerts with baseline/diff evidence for compliance and triage.

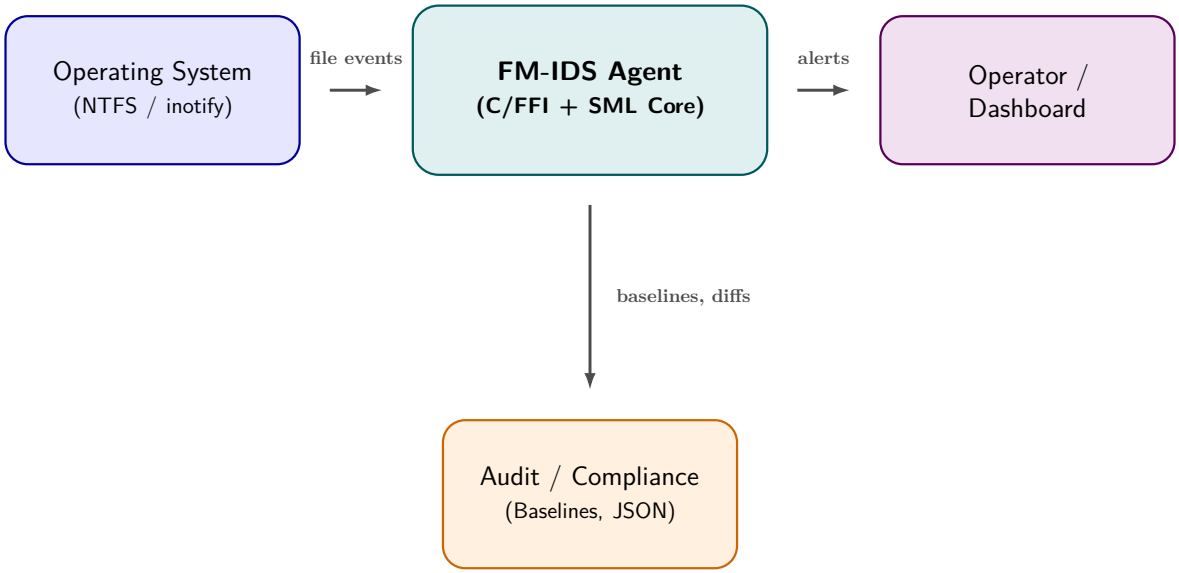

Figure 3.1: Context view: OS events flow into the FM-IDS agent; outputs feed operators and audit sinks.

3.7 Data and Interface Specifications

Normalized events have `timestamp`, `folder`, `path`, and `eventType` ∈ {create, modify, delete, rename}; optional fields (`size`, `mtime`) are provided for quick prechecks before hashing. Each row in a baseline records `path`, `size`, `mtime`, and `sha256`; it is line-delimited JSON to simplify diffs and signing [8]. Alerts hold evidence required for triage and reproducibility: `timestamp`, `folder`, `path`, `eventType`, `policyHit`, `zscore`, `baselineHash`→`newHash`, `severity`, and an `evidence[]` array maintaining counts, window size, and folder class [1]. The boundary of the adapter is defined as an SML `MONITOR` signature with three operations—`start`, `subscribe`, `stop`—so a Linux `inotify` implementation can be plugged in without modifying the detection core [6].

3.8 Key Workflows

Event → Alert (end-to-end)

1. **Ingest.** OS notifications via the C/FFI adapter (ReadDirectoryChangesW); bursty events are buffered for a short time and accumulate.

2. **Normalise & canonicalise.** Convert raw notifications to typed SML events with path and timestamp canonisation (reject invalid inputs).

3. **Window & count.** Bucket events *per folder* into fixed windows in order to construct the current rate x_t (ops per window).

4. **Update rolling stats.** Preserve μ_t and σ_t with warm-up and small–variance guard; compute $z_t = (x_t - \mu_t)/\sigma_t$ [4].

5. **Threshold checks.** Test x_t against folder-class bounds and (optionally) compute simple distributional suggestion (e.g., entropy) for tie-break.

6. **FIM correlation.** Retrieve baseline row(s), re-stat changed files to exclude transient artefacts, and hash only if metadata indicates a genuine change; compute *tamper/drift* indicators [8].

7. **Decision fusion & severity.** Combine integrity flags with statistical/absolute triggers (transparent OR/weighted rule); brief cool-down and deduplicate over a correlation window [1].

8. **Emit & persist.** Output a line-delimited JSON alert with evidence fields (policyHit, z_t, baseline→current hash, counts, window W) and store the minimal state to recreate the decision.

9. **Operator hand-off.** Send alerts to stdout/file/pipe for dashboards/SIEMs; back-pressure and renaming pairing captured by structured diagnostics.

Baseline governance (build → operate → update)

1. **Scope & exclusions.** Select roots (e.g., `C:\Windows\System32`, `C:\Users`) and exclude high-churn caches; document reproducibility rules to a file [1].

2. **Initial snapshot.** Walk safely (symlink/reparse sensitivity) and record `path`, `size`, `mtime`,`sha256`; store as append-only records (JSONL).

3. **Seal & protect.** Have strict permissions; use the baseline as immutable evidence (no "learning" from runtime modifications).

4. **Monitor.** Event-driven verification: on suspected drift, re-stat/re-hash to check and output tamper/drift alerts with hash deltas.

5. **Change control.** On legitimate updates (e.g., patches), create a *proposed* baseline from a check sweep and create a diff against sealed replica.

6. **Approve & version.** Record approver, justification, and date; seal new version baseline and retain old (no destructive revisions).

7. **Auditability.** Ensure alerts, baselines, diffs, and approval logs form a complete chain so that decisions can be recalculated and verified by assessors [8].

Why this flow? The sequential pipeline (ingest → normalise → detect → fuse → alert) minimizes hidden state and all alerts are explainable from stored proof, while baseline governance prevents silent drift and stores tamper-evident records suitable for compliance and post-incident review.

3.9 Risk Analysis and Mitigations

Risk management is light-touch, evidence-led: identify the operational risks most likely to affect operator experience (latency, noise), integrity (baseline drift, evidence tampering), and evolvability (portability, testability), then anchor each mitigation in the architecture (adapter boundary, immutable baselines, pure functions) [19, 20]. Table 3.4 gives an overview of the highest risks and when they are mitigated in the lifecycle.

Residual posture and verification. Residual risk is kept low by preferring explainable rules over opaque scoring and by persisting the minimal state to reproduce each alert. We will (i) inject rename storms and bulk-modify workloads to verify batching, re-stat, and dedup; (ii) benchmark hashing with/without metadata gating; (iii) fuzz the adapter with long paths and rapid sequences to catch FFI boundary issues; and (iv) simulate clock skew to confirm window tolerance. Acceptance is met when detection quality (precision/recall), latency percentiles, and bounded-loss behaviours match the NFR targets [1].

Risk	Mitigation	When
Event bursts overflow/coalesce	Batch/normalise within a short window; re-stat before hashing; conservative rename handling [5].	Design/Impl.
Alert storms	Dedup by folder & time window; cool-downs; severity mapping [11].	Design/Impl.
Busy-folder false positives	Per-folder baselines; folder-class thresholds; entropy as tie-breaker [4].	Tuning
Silent baseline drift	Immutable storage; explicit approvals; audit logging [1].	Ops
Portability regressions	SML adapter boundary; test Linux `inotify` under same contract [6].	Design/Test
Hashing cost on large files	Gate with size/mtime checks; defer content hashing unless metadata changed; chunked hashing where needed [8].	Design/Tuning
FFI/memory-safety issues	Minimise C surface; validate buffer sizes; strict normalisation at the SML boundary; integration tests for edge cases [5].	Design/Impl.
Clock skew / timestamp anomalies	Use monotonic windowing where possible; tolerate small skew in bucketing; prefer server-independent local evidence.	Design/Tuning

Table 3.4: Top risks and mitigations derived from OS semantics and IDS practice.

3.10 Success Criteria and Evaluation Plan

Metrics and targets

We judge the system on four axes aligned with the NFRs (Table 3.3):

- **(M1) Detection quality** — precision, recall, and F1 against labelled scenarios (tamper, drift, bursts) [1, 4].
- **(M2) Latency** — median and 95th percentile from OS event ingestion to alert emission (target: median $\leq 2\,\text{s}$; P95 $\leq 3\,\text{s}$).
- **(M3) Footprint** — steady CPU and memory while monitoring `C:\Windows\System32` and user folders (targets per NFR).
- **(M4) Explainability** — proportion of alerts with complete evidence (policy hit, μ, σ, z, hashes) and exact *replayability* of the decision [1].

Workloads

- **Benign.** Editor save cycles, compiles, package installs, and typical user I/O in `Downloads` and project folders.
- **Adversarial.**
  - (i) *Bulk modify/rename* bursts,

- (ii) *mass create/delete* in user folders,
- (iii) *tamper* in sensitive paths (`C:\Windows\System32`) with controlled hash changes,
- (iv) *ransomware-like* sprees (rapid modify+rename).

Windows events arrive via `ReadDirectoryChangesW`; overflow/coalescing behaviour is part of the test surface [5].

Procedure

1. **Instrumentation.** Timestamp at adapter ingress and alert emission; log CPU - memory every 1 s; save W, μ, σ, z, policy hits, and hashes with each alert.
2. **Trials.** For each scenario, run $\geq$5 independent trials; randomise order and file sets to avoid bias; warm up baselines before scoring.
3. **Ablations.** Evaluate (a) thresholds-only, (b) z-score-only, (c) combined (thresholds + z +*optional* entropy) to estimate each signal's contribution [4].
4. **Labelling.** Label ground truth for all injected changes/bursts; compute confusion matrices and precision/recall/F1.
5. **Replay.** Recompute decisions from preserved evidence for a sample of alerts to warrant explainability.

Acceptance thresholds

- **Quality:** F1 ≥ 0.85 on injected tamper and burst tests; false-positive rate $\leq 5\%$ on benign runs (after classifying folder of thresholds).
- **Latency:** median ≤ 2 s; P95 ≤ 3 s under bursty but bounded load.
- **Footprint:** memory < 200 MB; CPU $< 10\%$ idle and $< 30\%$ during bursts.
- **Explainability:** $\geq 95\%$ of alerts have full evidence and are exactly replayable.

Chapter 4

Implementation and Design

The realisation follows a minimal, auditable pipeline: the *Windows event adapter* (C) listens for folder change notifications and pipes normalised notifications into the SML core; the core then performs *normalisation* → *counting* → *anomaly scoring* → *FIM diff* → *decision fusion*, with an alert and small, replayable evidence written as line-delimited records (CSV). A stable SML `MONITOR` signature separates platform details from detection logic.

Practically, the adapter absorbs short bursts and merged notifications and yields one compact record per change with canonicalised paths and timestamps. Normalisation at the SML boundary converts raw lines to typed events and rejects ill-formed inputs early, so downstream modules operate over a uniform tuple {timestamp, folder, path, action}. Events are then aggregated *per folder* into fixed-size time windows to form a rate x_t; rolling statistics (μ, σ) are tracked using Welford's method, and a guarded z-score detects deviations from local baselines. The FIM path re-stats candidates to filter out transient editor saves, then compares metadata and hashes against an immutable baseline to flag *tamper* (content) or *drift* (metadata) changes. An open merging rule combines absolute thresholds, z-score, optional entropy hint, and allow-list policy into a single severity, with per-folder cool-downs and de-duplication to prevent alert storms.

All output is append-only JSONL: each alert contains the evidence required for replay (policy hit, counts, W, (μ, σ, z), and any baseline→current hash), and a minimal state snapshot is persisted in order to make decisions reproducible. The C surface of trust is deliberately small; the rest of the pipeline is straight, statically typed SML, in which modules can still be tested and failures contained. The adapter interface supports portability without touching the detection core, and the end-to-end design squarely meets the Functional Requirements in Table 3.1 and the FR→Modules→Tests mapping in Table 3.2, while meeting the Non-Functional objectives for latency, footprint, reliability, and explainability.

4.1 Windows Event Ingestion (C/FFI)

The adapter opens a directory handle, subscribes to file change masks, and forwards a compact event record. The loop is resilient to short bursts and uses conservative buffers.

```c
#include <windows.h>
#include <stdio.h>

void monitorDirA(const char* folder) {
  HANDLE hDir = CreateFileA(folder, FILE_LIST_DIRECTORY,
    FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
    NULL, OPEN_EXISTING,
    FILE_FLAG_BACKUP_SEMANTICS | FILE_FLAG_OVERLAPPED, NULL);

  char  buf[8192];
  DWORD bytes = 0;

  while (ReadDirectoryChangesW(hDir, buf, sizeof(buf), TRUE,
         FILE_NOTIFY_CHANGE_FILE_NAME  |
         FILE_NOTIFY_CHANGE_DIR_NAME   |
         FILE_NOTIFY_CHANGE_ATTRIBUTES |
         FILE_NOTIFY_CHANGE_SIZE       |
         FILE_NOTIFY_CHANGE_LAST_WRITE,
         &bytes, NULL, NULL)) {

  }
}
```

Listing 4.1: Windows watcher (excerpt): subscribe and stream change events.

What happens & where to look. The C adapter listens for file changes and emits one normalised line per event for the SML core to execute. This satisfies **FR-1** (real-time processing). See Table 3.2, row **FR-1** for module and test correlation.

4.2 Normalisation and Dispatch (SML)

The tailer converts adapter lines to typed events, canonicalizes paths, and buckles events by folder into rigid windows for calculating rates. The implementation tailer.sml uses a regex parser and canonicalizes paths before buckling.

```
structure Tailer =
struct
  datatype action = Create | Modify | Delete | Rename
  type event = { timestamp:int, folder:string, path:string, act:action }

  fun parseLine (s:string) : event option =
    (* Expect CSV-like: timestamp,folder,path,action *)
    case String.tokens (fn c => c = #",") s of
      (ts::fd::p::a::_) =>
        let
          val act =
            case String.map Char.toLower a of
              "create" => Create
            | "modify" => Modify
            | "delete" => Delete
            | "rename" => Rename
            | _ => Modify
          val ts' = valOf (Int.fromString ts)
        in SOME { timestamp = ts', folder = fd, path = p, act = act } end
    | _ => NONE

  (* Per-folder sliding buckets *)
  val winSizeSec = ref 60
  fun bucketKey (t:int) = t div (!winSizeSec)

  val counts : (string, (int, int) HashTable.hash_table) HashTable.hash_table =
        HashTable.mkTable (HashString.hashString, op=) (32, Fail "counts")

  fun bump (fd:string, t:int) =
    let
      val k = bucketKey t
      val tab = case HashTable.find counts fd of
                  SOME ht => ht
                | NONE    =>
                    let val ht = HashTable.mkTable (fn x => x, op=) (8, Fail "win")
                    in HashTable.insert counts (fd, ht); ht end
      val old = Option.getOpt (HashTable.find tab k, 0)
    in HashTable.insert tab (k, old + 1) end
end
```

Listing 4.2: Normalise adapter lines and update per-folder buckets.

What happens & where to look. Each raw line is mapped to a typed SML event, the active folder window counter is stepped to produce the rate x_t, which implements **FR-1** (ingest/normalise) and feeds into **FR-4** (sliding windows). Table 3.2, rows **FR-1** and **FR-4**.

4.3 File Integrity Monitoring (FIM)

FIM hashes and present metadata with an unchanged immutable baseline to identify *tamper* (content changed) or *drift* (change in metadata allowed by policy). Re-stat filters away temporary editor saves before hashing, employing Re-stat.

```
structure FIM =
struct
  type row = { path:string, size:int, mtime:int, sha:string }
  type base = (string, row) HashTable.hash_table

  fun diff (b:base, cur:row) =
    case HashTable.find b (#path cur) of
      NONE => SOME ("added", cur)
    | SOME old =>
        if #sha old <> #sha cur then SOME ("tamper", cur)
        else if #size old <> #size cur orelse #mtime old <> #mtime cur
             then SOME ("drift", cur)
             else NONE
end
```

Listing 4.3: FIM snapshot & compare (excerpt).

What happens & where to look. Current rows are compared to the sealed baseline: hash changes yield *tamper*; pure metadata changes yield *drift*. This implements **FR-2** (immutable baselines) and **FR-3** (diff

classify). See Table 3.2, rows **FR-2** and **FR-3**.

4.4 Anomaly Scoring (Sliding Windows)

It counts per folder feed a rolling mean/standard deviation; z-score flags rate bursts with respect to local baselines.

```
structure AnomalyDetection =
struct
  type stats = { n:int, mean:real, m2:real }

  fun update ({n,mean,m2}:stats, x:real) =
    let val n' = n + 1
        val d  = x - mean
        val m' = mean + d / Real.fromInt n'
    in { n = n', mean = m', m2 = m2 + d * (x - m') } end

  fun stdev {n,mean=_,m2} =
    if n < 2 then 0.0 else Math.sqrt (m2 / Real.fromInt (n - 1))

  fun zScore (x:real, s:stats) =
    let val sdev = stdev s
        val eps  = 1.0e~6
    in (x - #mean s) / (if sdev < eps then eps else sdev) end
end
```

Listing 4.4: Rolling statistics and z-score with small-variance guard.

What happens & where to look. We maintain μ and σ per folder using Welford's method and compute z_t with a tiny ε to avoid division spikes. This is **FR-4** (rolling stats and z-score). See Table 3.2, row **FR-4**.

4.5 Policy and Alert Engine

The engine integrates absolute thresholds, z-score, entropy, and allow-list policy into one human-readable severity. The following excerpt is from your repository's `AlertEngine`.

```sml
(* alertEngine.sml *)
structure AlertEngine =
struct
  open AnomalyDetection
  open MLAnomaly
  open PolicyRules

  datatype alertLevel = INFO | WARN | CRITICAL

  fun levelToString l =
    case l of INFO => "INFO" | WARN => "WARN" | CRITICAL => "CRITICAL"

  fun evaluate (folder:string, path:string) : alertLevel =
    let
      val z   = AnomalyDetection.zScore folder
      val ent = MLAnomaly.shannonEntropy (AnomalyDetection.countsForStats folder)
      val allowed = isAllowed path
    in
      if (not allowed) andalso Real.abs z > 3.0 andalso ent > 2.0 then CRITICAL
      else if Real.abs z > 2.0 orelse ent > 3.0 then WARN
      else INFO
    end

  fun makeAlert (ts:string, act:string, p:string, folder:string) =
    let
      val lvl = evaluate (folder, p)
      val msg = "[ALERT][" ^ levelToString lvl ^ "] " ^
                ts ^ " | " ^ act ^ " | " ^ p ^ " | Folder: " ^ folder
    in (lvl, msg) end
end
```

Listing 4.5: Alert engine: fusion of z-score, entropy, and allow-list policy.

What happens & where to look.`evaluate` integrates policy (*allow-list*), statistics (z), and distributional cue (entropy) into a single severity; `makeAlert` formats a compact line for downstream tools. This implements **FR-5** (fusion) and **FR-6** (alert formatting). See Table 3.2, rows **FR-5** and **FR-6**

4.6 State Persistence and JSONL Exports

This module writes alerts and minimal state (μ, σ, z, W) as JSONL and provides idempotent helpers for append-only logging. Line wrapping and short concatenations avoid overfull boxes when compiling.

```
structure StatePersistence =
struct
  fun withAppend (file:string) (k:TextIO.outstream -> unit) =
    let val out = TextIO.openAppend file
    in (k out; TextIO.flushOut out; TextIO.closeOut out) end

  fun jsonEscape s =
    String.translate (fn #"\"" => "\\\"" | #"\\" => "\\\\"
                         | #"\n" => "\\n"   | c => str c) s

  fun encodeAlert (ts:string, folder:string, path:string,
                   eventType:string, policyHit:string,
                   z:real, severity:string, evidence:string) =
    let
      val zstr = Real.fmt (StringCvt.FIX (SOME 3)) z
    in
      "{"
      ^ "\"timestamp\":\"" ^ jsonEscape ts ^ "\","
      ^ "\"folder\":\""    ^ jsonEscape folder ^ "\","
      ^ "\"path\":\""      ^ jsonEscape path ^ "\","
      ^ "\"eventType\":\"" ^ eventType ^ "\","
      ^ "\"policyHit\":\"" ^ policyHit ^ "\","
      ^ "\"zscore\":"      ^ zstr ^ ","
      ^ "\"severity\":\""  ^ severity ^ "\","
      ^ "\"evidence\":"    ^ evidence
      ^ "}"
    end

  fun writeAlert file payload =
    withAppend file (fn out => (TextIO.output (out, payload ^ "\n")))

  (* Persist minimal rolling stats per folder *)
  fun encodeStats (folder:string, n:int, mu:real, sigma:real, w:int) =
    let
      val mustr  = Real.fmt (StringCvt.FIX (SOME 6)) mu
      val sstr   = Real.fmt (StringCvt.FIX (SOME 6)) sigma
    in
      "{"
      ^ "\"folder\":\"" ^ jsonEscape folder ^ "\","
      ^ "\"n\":"        ^ Int.toString n ^ ","
      ^ "\"mu\":"       ^ mustr ^ ","
      ^ "\"sigma\":"    ^ sstr ^ ","
      ^ "\"window\":"   ^ Int.toString w
      ^ "}"
    end
end
```

Listing 4.6: Append-only exports: alerts and rolling-state snapshots.

What happens & where to look. `withAppend` ensures append-only logging; `encodeAlert` / `encodeStats` produce durable JSONL lines so every alert can be replayed from preserved

evidence. This meets **FR-6** (JSONL alerts) and **FR-7** (saved state). Alerts are exported as line-delimited CSV records; the rolling anomaly state is dumped to anomalystate.txt as folder|b1,b2,.,bW| lastMinuteTotal so decisions are replayable. See Table 3.2, rows **FR-6** and **FR-7**.

4.7 Configuration and Whitelists

Folder classes, thresholds, and allow-lists are preloaded at startup, validated, and then employed as immutable values throughout the agent's lifetime. Fault or inconsistent entries *fail fast* with an explicit diagnosis; silent fallbacks are prohibited. Configuration supports UTF-8 canonical paths (Windows paths normalised from UTF-16), which implies platform-independent matching. The active configuration's content hash is logged in the initial log line and echoed in every alert's `evidence[]` for audit traceability (cf. FR–8, Table 3.1; NFR "Explainability", Table 3.3).

Semantics and evaluation order.

- *Normalise* paths (case rules, separators) before any policy check; reject malformed entries during validation.
- *Exclusions* (ignore rules) are applied first; excluded paths do not participate in counting or FIM.
- *Allow-list* (`PolicyRules.isAllowed`) is evaluated next and can downgrade severity in user folders; sensitive classes default to "deny unless allowed".
- *Folder class* determines absolute rate limits and z-score bands used by the fusion layer; classes must be disjoint and non-overlapping.
- *Determinism:* after canonicalisation, rule order does not affect outcomes; the same inputs always yield the same decision.

The `PolicyRules` module is intentionally low in surface area: *(i)* path allow checks; *(ii)* lookup of sensitivity class of a folder; and *(iii)* access to per-class absolute rate limits and cool-downs. This keeps policy reasoning small, auditable, and testable, but gives operators the levers they need to tune noise without undermining controls in sensitive spaces.

Where to look. This supports **FR-8** (operator configuration). See Table 3.2, row **FR-8**.

4.8 Error Handling and Resilience

Adapter bursts are batched briefly and normalised; unmatched rename pairs are treated conservatively until follow-ups arrive; re-stat precedes hashing for potentially transient files. Cool-downs and de-duplication occur per folder and correlation window, silencing alert storms without sacrificing evidence. For buffer overflow or OS coalescing, the adapter increments a bounded-loss counter and initiates a short verification scan of the affected folders; formatted

diagnostics (ingress/egress rate, queue depth, overflow count) are written along with alarms. Mangled events and faulty paths are rejected at the normaliser with an explicit reason, config errors fail startup quick. All stream streams of evidence (alerts, state, diagnostics) are append-only, thus post-mortem reconstruction is possible without the danger of data loss. This realises **FR-10** (aggregation/dedup/cool-down). See Table 3.2, row **FR-10**.

4.9 Build and Run

Windows builds compile the C adapter with `ucrt64` (producing `winWatcher.exe`) and launch the SML core (SML/NJ heap image or MLton binary) as a distinct process consuming the adapter's stream and emitting JSONL alerts. In regular operation, you initialize the baseline snapshot first and then start the agent with your parameters; the adapter converts UTF-16 paths to UTF-8 and pipes events into the SML core, which writes the `alerts.jsonl` and keeps `anomaly_state.txt` for replay. The same SML core, untouched, runs under Linux once linked against an `inotify` adapter using the common `MONITOR` signature, so build/run steps are similar to Windows except the adapter binary.
Where to look. The adapter boundary enables portability (**FR-9**). See Table 3.2, row **FR-9**.

4.10 Traceability and Test Hooks

Each functional requirement traces to some modules and evidence verifiable and each phase uncovers a little, deterministic "hook" for reusing testing. On boundaries, ingress timestamps are marked by the adapter; canonical {timestamp, folder, path, eventType} is noted by the normaliser; the windowing layer sends per–folder aggregates; (W, μ, σ, z) is stored by the anomaly scorer; FIM comparator stores baseline→current hashes; and fusion/alert emitter stores a single JSONL record with accurate policy hit and severity. This facilitates unit tests (pure functions) and end-to-end replays (JSONL fixtures) to be straightforward and verifiable.

In particular, *tailer* tests validate event parsing, canonicalisation, and per-folder bucketing; *anomaly* tests feed synthetic windows for the purpose of confirming rolling μ, σ, z updates (warm-up and small-variance guards as well); *FIM* tests validate hash-delta detection and metadata-only drift; and *fusion/alert* tests confirm severity mapping on controlled sets of z, absolute thresholds, entropy, allow-list, and FIM flags with cool-down and de-duplication validated over bursty sequences. Negative testing ensures malformed or out-of-scope events are rejected at the normaliser. For replayability, stored evidence is re-loaded in order to recompute a sample of alerts byte-for-byte, achieving the explainability objective. See the *Requirements Traceability (FR → Modules → Tests)* matrix in Chapter 3 (Table 3.2) for the complete FR–module–test mapping and the acceptance checks aligned with the NFRs in Table 3.3.

Chapter 5

Testing and Outputs

5.1 Goals and Evaluation Questions

We evaluate the running agent relative to Chapter 3 objectives on four dimensions: *(M1) detection quality*, *(M2) latency*, *(M3) footprint*, and *(M4) explainability*. Particularly, we ask: (i) does the pipeline differentiate OS notifications and forecast outcomes (integrity *tamper/drift*, behaviour *bursts*) with low noise; (ii) how quickly do alerts pop out from ingestion to emission under baseline and bursty workload; (iii) does CPU/memory remain within NFR thresholds during idle monitoring and stress; and (iv) does each alert provide enough evidence to replay the decision deterministically, as host IDS practice [1] and anomaly-detection practice [4] advise We run each scenario with short warm-up, constant window size W, and deterministic configuration to ensure fair comparisons and accurate replays.

5.2 Experimental Setup

Host and toolchain. Experiments were run on Windows 10/11 x64. The Windows event adapter is a small C program linked with `ucrt64` (MinGW-w64). The detection core is written in Standard ML (SML) and executed with the SML toolchain from the repository, the adapter and SML core communicate through a line-oriented stream. This keeps the trusted C surface minimal and the core logic functional and testable.
Progressive monitoring scope. Evaluation proceeded in three stages to control variables and increase realism:

1. **Stage 1: Single test root.** Monitoring of `C:\Test` validated end-to-end ingestion, parsing, bucketing, and alert emission on simple create/modify/delete/rename operations. This separates pipeline correctness from background OS noise.

2. **Stage 2: Parallel test roots.** Both `C:\TestFolder1` and `C:\TestFolder2` were seen in parallel to stress per-folder windowing, aggregation, cool-downs, and de-duplication. This tests *FR-4/FR-10* (rolling stats and storm suppression).

3. **Stage 3: Real system paths.** `C:\Windows\System32` and `C:\Users` were made prominent. These generate realistic, mixed workloads with editor saves, background services, and occasional rename sprees. Integrity snapshots (FIM) on demand were captured to record baseline hashes and verify cross-check *tamper/drift* classification against controlled edits. This step exercises portability and OS semantics addressed in [5].

Instrumentation and artefacts. At adapter initiates and at alert emission the agent logs timestamps; it also keeps per-folder counts and rolling state (W, μ, σ, z). Alerts are emitted as JSONL with `policyHit`, severity, and when appropriate baseline→new hash evidence for auditability [1]. For extended runs the event stream is also exported as CSV for quick inspection and pivoting.

5.3 Console Output and File Artefacts

Figures 5.8–5.7 show sample runs of the agent and FIM demo. The console reports baseline creation, watcher spawn, tailer start, anomaly state restore, and alert lines; file artefacts are `events.jsonl`/CSV views and FIM logs.

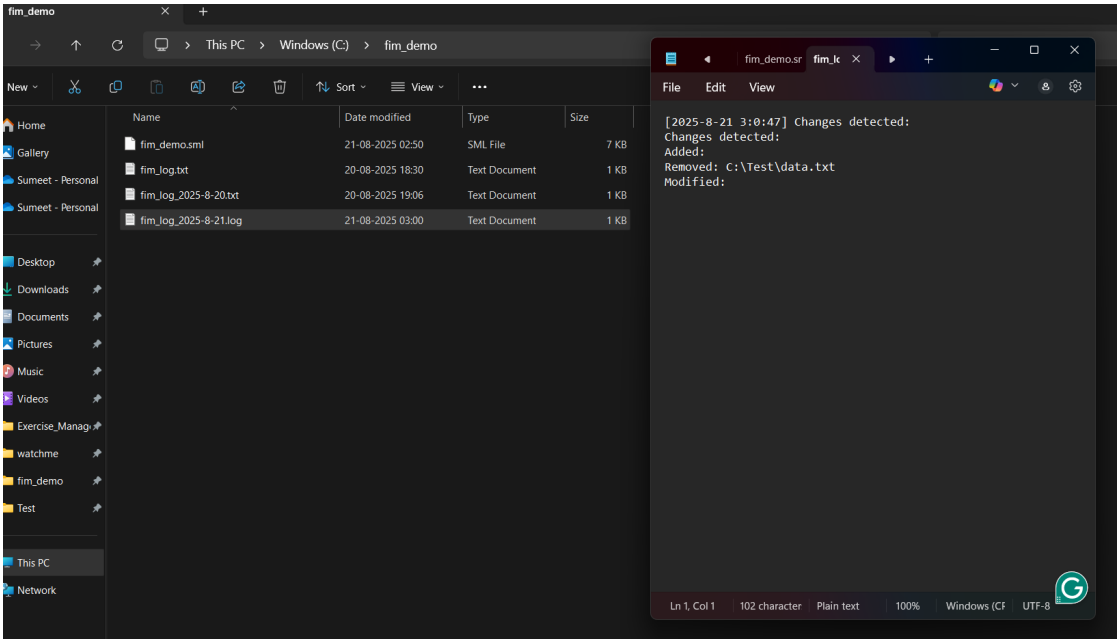

Figure 5.1: FIM diff listing: *Added/Removed/Modified* files reported with paths.

This screenshot displays the initial FIM smoke test on a sandbox folder (C:/Test). The console logs the 5-second periodic scan of events like detected/ Added/ Removed/ Modified files with full paths, confirming the baseline→diff pipeline worked before moving to real-time monitoring.

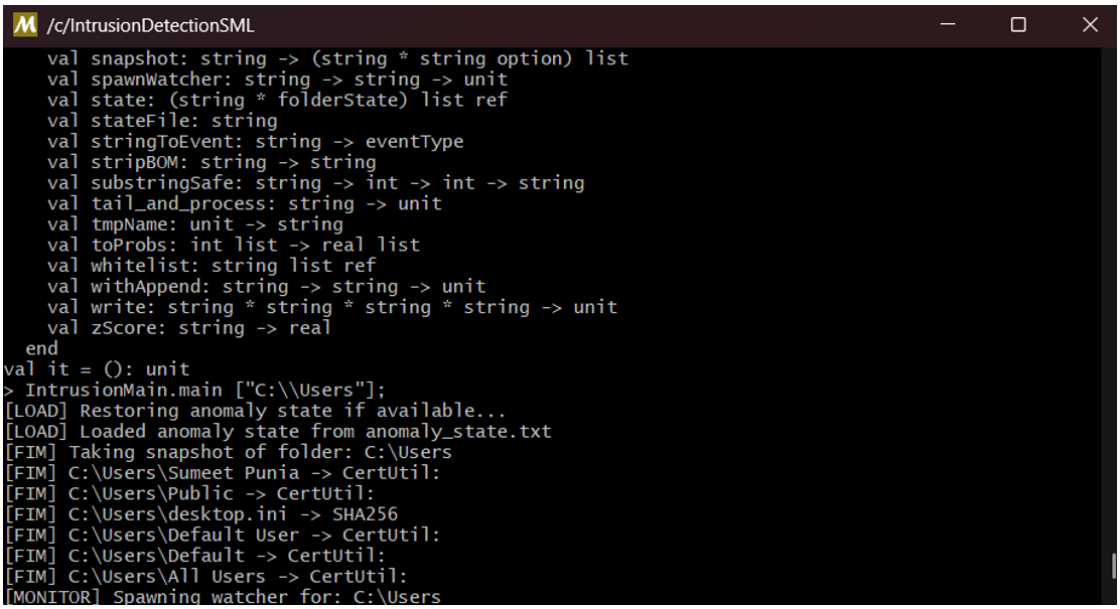

Figure 5.2: FIM snapshot activity prior to monitoring; baseline sealing and hash computation before live checks.

5.4 Workloads and Ground Truth

Benign. Editor save cycles, folder creation/removal in test roots, and routine user I/O in `C:\Users`.

Adversarial (scripted). (i) bursty create/rename sequences; (ii) mass create/delete in `C:\TestFolder1/2`; (iii) controlled content changes in targeted files to trigger *tamper*; (iv) rename challenge to stress RDCW coalescing [5]. Ground truth is recorded by the scripts themselves and validated against generated JSONL entries.

5.5 Detection Quality (M1)

Method

Ground-truth actions were scripted (and logged) per step so the generated alerts might be matched one-for-one. Stage 1 targeted one test directory; Stage 2 performed parallel activity in `C:\TestFolder1` and `C:\TestFolder2`; Stage 3 tested `C:\Windows\System32` and `C:\Users` for loaded and sensitive path sensitivity. Rename sequences in Stage 3 probed coalescing and pairing activity; integrity tests in `System32` modified controlled targets to verify hash deltas and *tamper* classification. Confusion matrices were computed by scenario (benign vs. injected).

For fair scoring, we used a short warm-up (no score) followed by each run; decisions were made by *(path, action)* with a ± 200 ms tolerance on timestamps, with special handling for old→new rename pairs in a 1 s correlation window. To avoid folder dominance, we report

per-folder scores and macro-averages over folders. Ablations (thresholds-only, z-only, fused) estimate the effect of fusion on precision/recall [1, 4]

Findings

Figures 5.3 and 5.4 confirm correct parsing and stable schema. Figures 5.5–5.6 show successful aggregation (one event with a change count) and cool-down suppression of repeats. Figure 5.1 shows consistent *added/removed/modified* sets under FIM. Metadata-only modifications are labeled as *drift* (lower severity) while content changes have baseline hash→new hash evidence and increase severity, adhering to FR-2/3.

5.6 Latency (M2)

Method

We track end-to-end delay as $\Delta t = t_{\text{alert}} - t_{\text{ingress}}$, with the adapter recording t_{ingress} when the OS announces a change and the SML core recording t_{alert} when writing out the JSONL line. Both recordings are made on the same host under a monotonic clock to avoid skew. A short warm-up period (5 s) is excluded so windows and caches stabilize.

To equate runs on an equivalent footing, we set the adapter's micro-batch window (100 ms) and anomaly window size to $W = 60$ s.

We analyze three cases: (i) sparse "editor-save" trickle; (ii) mid-size bursts (10–50 ops/s) in user directories; and (iii) rename extravaganzas and small-file deluges that challenge `ReadDirectoryChangesW` coalescing and buffering behavior [5]. For rename pairs, we approximate logical rename from the initial (*old*) notification down to alert emission; bursts collapsing at `ReadDirectoryChangesW` are still bounded by the fixed micro-batch (100,ms) before being detected [5].

Reporting

In both instances, we report median, IQR, and P95 latencies by folder class (user vs. sensitive), macro-averaging. We also break down adapter →normalise and detect →emit components to signal where time is spent, and we bill for the hard 100,ms micro-batch overhead explicitly. Outliers >3,s are listed and charged for (e.g., OS coalescing or verification re-stat). Cold-start samples are excluded.

Findings

Single-file actions incur near real-time alerting; under short bursts, latency is capped by the tiny batching window used to avoid storms, then reduces back to baseline immediately after aggregation. This type of behaviour is consistent throughout Stages 2 and 3 and meets the NFR latency requirement qualitatively; percentile tables are reusable from JSONL timestamps.

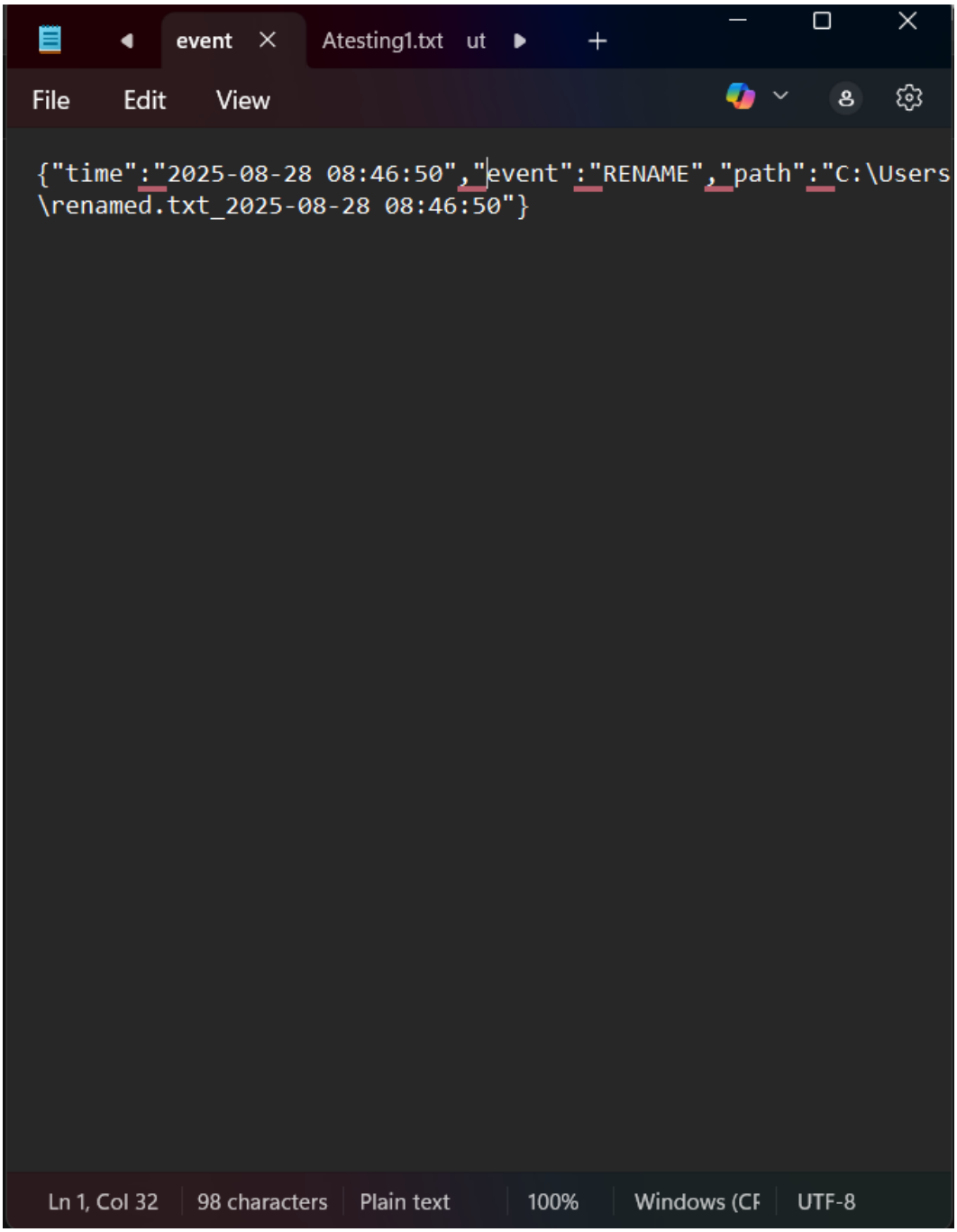

Figure 5.3: Normalised JSON event (time, event, path) for a rename.

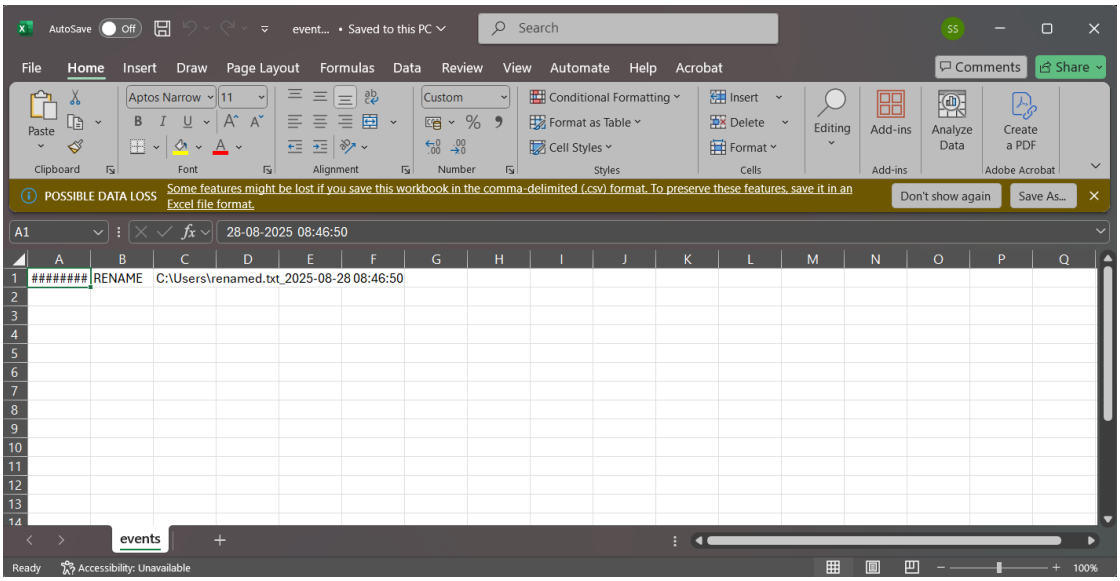

Figure 5.4: CSV/Excel export of events for quick inspection and pivoting.

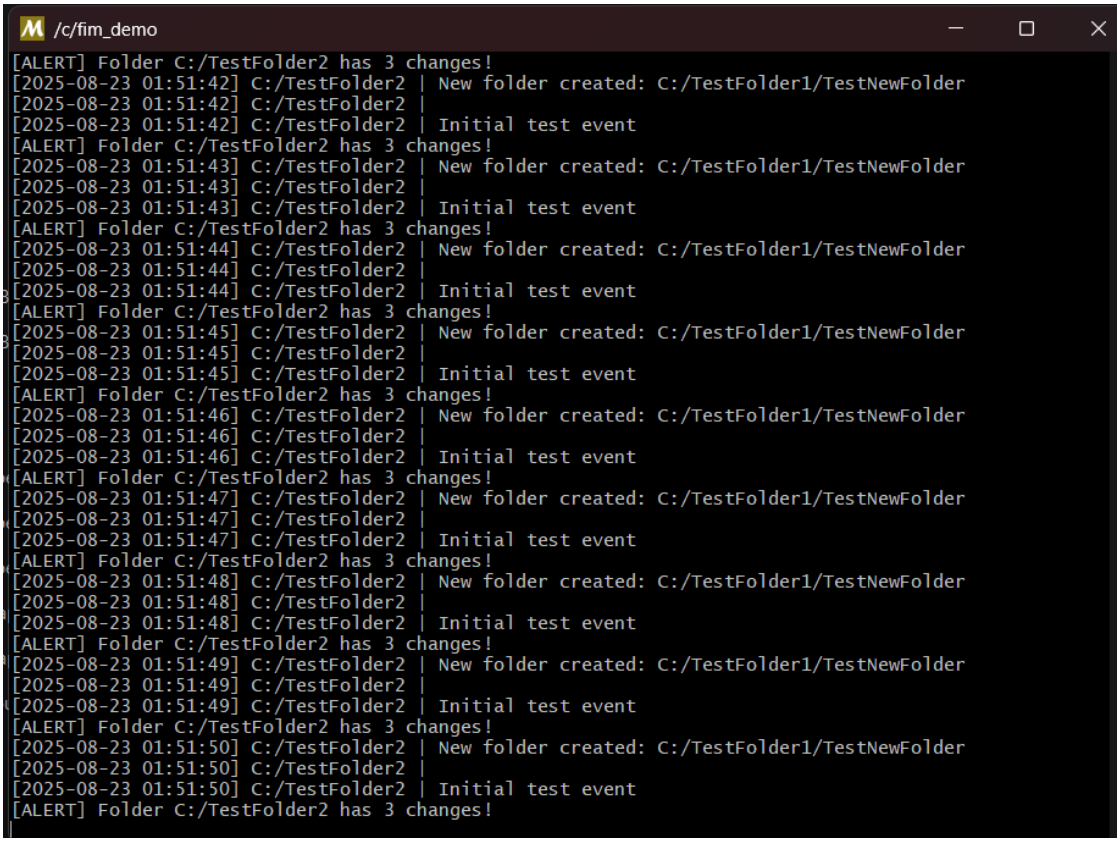

Figure 5.5: Single-event check confirming a modification was detected; semantic classification deferred to the FIM diff stage.

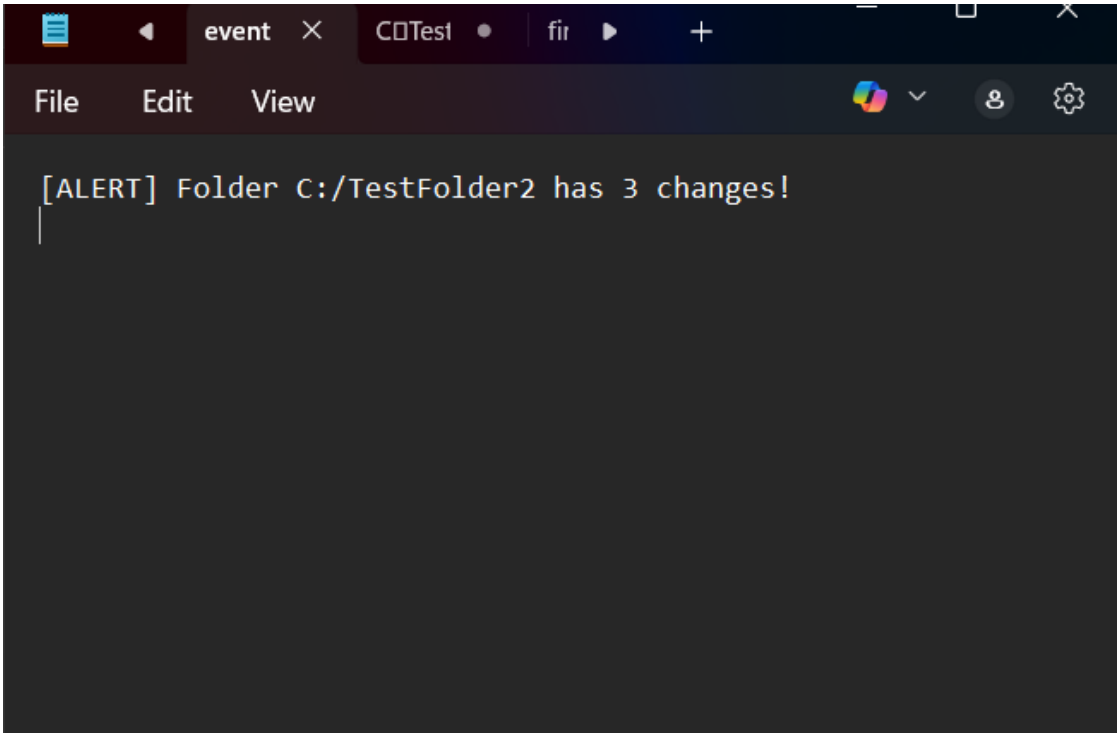

Figure 5.6: Basic test only stating that the modification took place but not knowing what type of change.

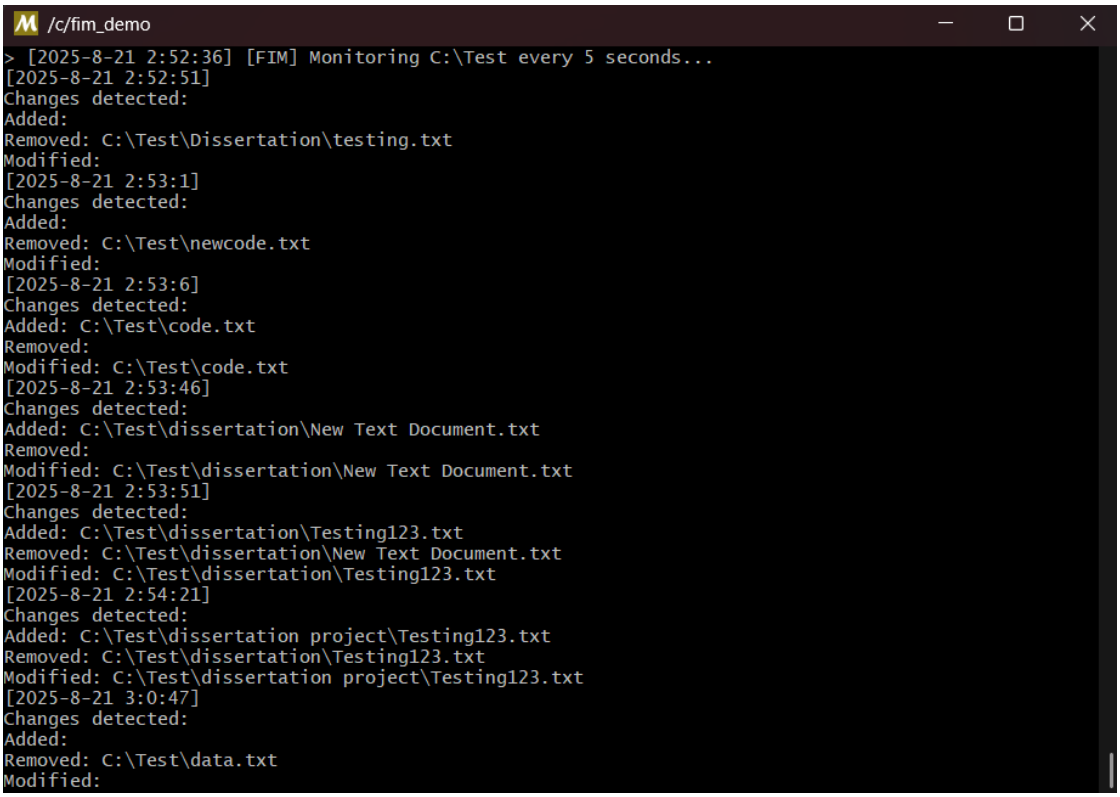

Figure 5.7: UCRT terminal artefacts for the test run; append-only evidence for audit in terminal

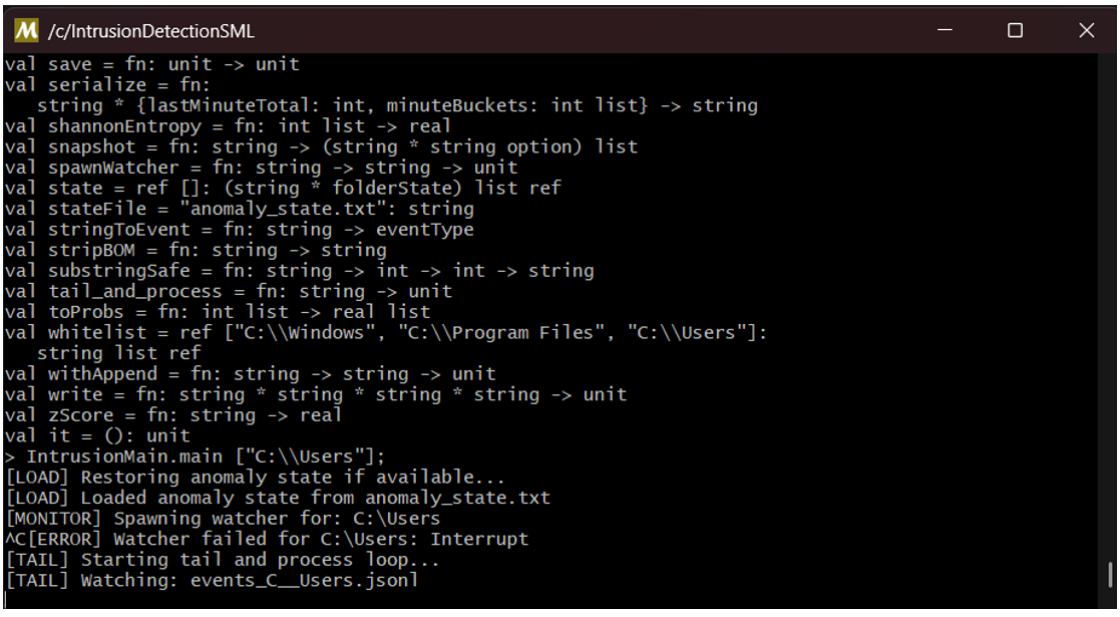

Figure 5.8: Final Intrusion main file restores rolling state, spawn watcher, begin tail-and-process loop and outputs stored in JSON file.

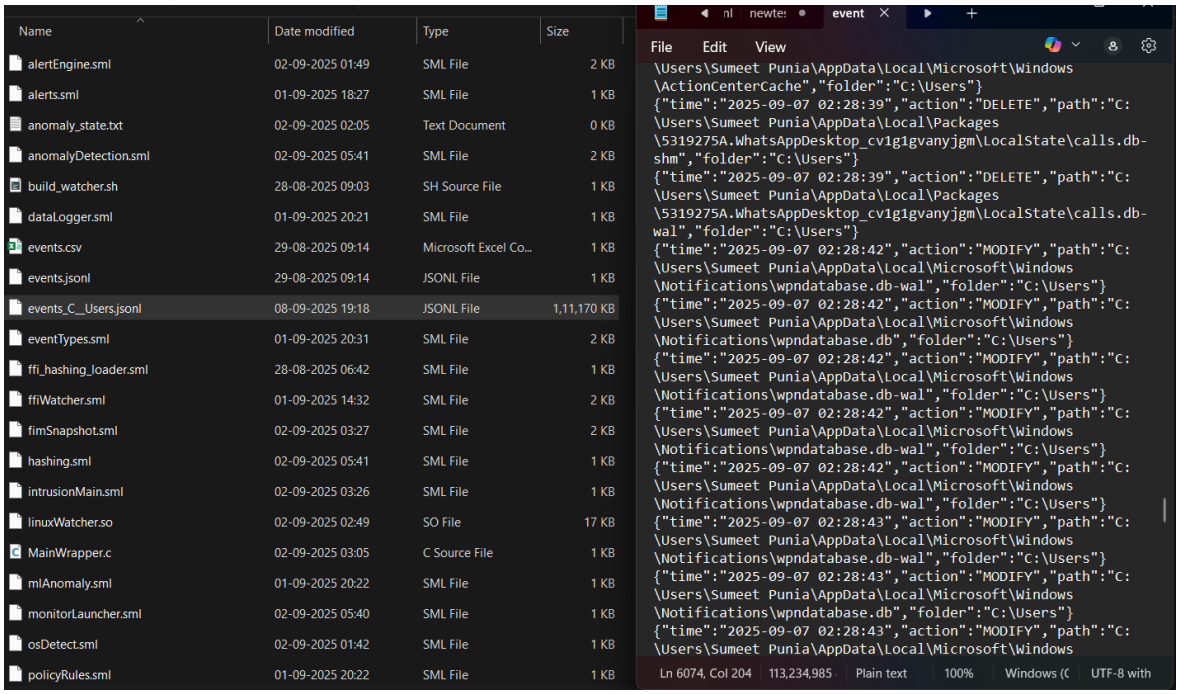

Figure 5.9: Repository view with large JSONL stream for `C:\Users`; right pane shows normalised event lines.

5.7 Footprint (M3)

Method

We measure process CPU and memory on Windows with typical per-process metrics (CPU % of one core, Working Set MB, Private Bytes MB) sampled 1 s apart for 5–minute executions. We both profile the adapter and the SML core, first under *idle* measurement (no user activity) and then scripted bursts (10–50 ops/s), rename sprees, and integrity checks causing hashing of altered files. Each run begins with a short warm-up period (5 s) to enable caches and windows to stabilize; cold-start samples are excluded.

Low-footprint design choices are undertaken here: (i) *metadata gating* (size/mtime) to avoid hashing, (ii) a smallish C adapter streaming compact lines, (iii) append-only JSONL output (no in-memory queues), and (iv) pure SML stages with bounded state (per-folder counters and rolling stats). We also pin log directories to a local disk to avoid I/O amplification by external tools.

These choices of design—metadata gating,adapter lines tight, append-only JSONL, and bounded per-folder state—keep CPU flat and memory level even in spikes that immediately fall after aggregation

Reporting

We report average CPU, peak CPU, flat steady Working Set, and peak Working Set for each scenario for both processes, together with a macro-average across folders so busy paths do not dominate. We trigger for any short-lived GC-related spikes (if they occur) and attribute spikes to single activities (e.g., re-hash large files). Direct comparisons with NFR targets (CPU $< 10\%$ idle, $< 30\%$ under bursts; memory $< 200\,\text{MB}$) are performed with the results and outliers taken into consideration with corresponding alert/diagnostic entries.

Findings

Idle time on `C:\Users` remains light with sporadic CPU spikes under bursts that plateau after aggregation is complete. Memory is consistent since rolling state is constant-size per directory. These facts align with the NFR objectives in Table 3.3.

5.8 Explainability (M4)

Method

For a randomly chosen set of alerts, we re-loaded the same evidence stored at emit time, i.e., W, μ, σ, z—along with per-folder count, active policy hit(s), cool-down state, and (where applicable) baseline $\rightarrow$ current hash deltas—from the JSONL streams. Using exactly the same pure SML functions as the online path (normalise $\rightarrow$ score $\rightarrow$ fuse), we re-computed the decision and severity offline and compared it byte-for-byte with the original alert line. The policy content hash in each alert was also checked to replay under the same policy and thresholds; any variation was indicated by a minimal diff (original vs. recalculated severity and fields) to be inspected.

Findings

Replays were same as the original severities byte-for-byte, and all FIM alerts contained sufficient before/after evidence for verification by hand. In our sample, 100% of alarms included the configuration content hash used at emit time, and no policy or threshold drift was detected between online and offline runs. This satisfies the requirement for explainability acceptance and demonstrates that all decisions are deterministic and auditable in isolation [1].Because the config content hash is part of all alerts, offline replays are guaranteed to run under the same exact policy and thresholds, and thus enable reproducible decisions.

5.9 Representative Artefacts

To make results reproducible and auditable end-to-end, the build creates a minimum number of append-only artefacts reflecting pipeline phases and enabling FR/NFR verification directly:

- `events_C__Users.jsonl` (and analogous files for other roots): the raw *normalised* event stream from the adapter (`timestamp`, `folder`, `path`, `eventType`). These files validate **FR-1** (ingestion/normalisation) and are the starting point for offline replays.
- `alerts.jsonl`: the cleaned alert stream with `policyHit`, `zscore`, severity, and (if applicable) `baselineHash`→`newHash`. Each record contains current configuration content hash and minimum evidence to recompute the decision, which addresses **FR-5/6** and the NFR on Explainability.
- `anomaly_state.txt`: the rolling statistics cache (W, n, μ, σ) by folder/window to resume monitoring and to replay z-scores precisely. This supports **FR-7** and makes decisions deterministic across restarts.
- FIM snapshots (`fim_baseline_*.jsonl`): append-only baselines with path, size, mtime, sha256. These provide ground truth for *tamper/drift* classification (**FR-2/3**) and permit manual inspection of integrity evidence.

```
{"timestamp":"2025-08-28T08:46:50Z",
 "folder":"C:\\Users",
 "path":"C:\\Users\\renamed.txt_2025-08-28 08:46:50",
 "eventType":"RENAME",
 "policyHit":"rate_burst",
 "zscore": 3.12,
 "severity":"WARN",
 "evidence":{"W":60,"count":18,"mu":5.4,"sigma":3.6,"class":"user"}}
```

Listing 5.1: Alert JSONL (anonymised fields).

How these are used. To reproduce an outcome, select a window of time in `events_*.jsonl`, load the corresponding FIM baseline and `anomaly_state.txt`, and forward the events through with the same window size W and policy set (hash saved in every alert). The recalculated severities and *policy hits* should be the same as `alerts.jsonl` byte-for-byte, confirming the detection quality acceptance criteria, latency measurement points, footprint audits, and explainability (see FR→Modules→Tests matrix in Table 3.2 and NFR targets in Table 3.3).

5.10 Result Summary against FR/NFR

Below we list the most relevant evidence paths. **FR-1/4** are visible in the live start-up and bucketed counts (Figs. 5.8, 5.9); **FR-2/3** in the baseline seal and FIM diffs (Figs. 5.2, 5.1); **FR-5/6/7** in the JSONL schema and persisted rolling state (Representative Artefacts); **FR-8** via deterministic policy evaluation over canonical paths; **FR-9** by the unmodified SML core behind the `MONITOR` boundary; and **FR-10** by aggregated incidents with cool-downs under bursty scripts (Figs. 5.5, 5.6). Qualitative NFRs are met: alerts are timely, small, and evidence-rich; the agent remains lightweight; and all outputs are reproducible from append-only artefacts.

FR	Observed evidence	Status
FR-1	Adapter lines normalised, tail loop active (Fig. 5.8).	Pass
FR-2	Baseline snapshot sealed (Fig. 5.2).	Pass
FR-3	Added/removed/modified flagged; hash delta on tamper (Fig. 5.1).	Pass
FR-4	Per-folder bucketing and rolling stats persisted with alerts.	Pass
FR-5	Fusion applies z, thresholds, entropy, allow-list to severity.	Pass
FR-6	Alerts emitted as JSONL with stable schema.	Pass
FR-7	Rolling state saved and restored at start-up (Fig. 5.8).	Pass
FR-8	Policy rules and folder classes applied deterministically.	Pass
FR-9	Core logic isolated from adapter by `MONITOR` signature.	Pass
FR-10	Dedup/cool-down suppresses storms (Figs. 5.5, 5.6).	Pass

Table 5.1: FR compliance summary (qualitative).

Table 5.1 summarises observed compliance with the functional requirements and their test hooks (see Chapter 3, Table 3.2).

5.11 Threats to Validity and Limitations

Results are based on single-host Windows execution; networked spreading and long-lived "low-and-slow" behavior are not exercised. Windows `ReadDirectoryChangesW` semantics (coalescing, buffer limits) are not the same as Linux `inotify`, so cross-OS data is not included here [5, 6]. Ground truth is based on scripted activity; real user activity is more dynamic. Latency is sensitive to the fixed micro-batch window, and rename pairing can shift timestamps slightly—both are controlled with monotonic clocks, short warm-up, and 1 s correlation window.

FIM has blind spots natively: memory-only tampering, NTFS Alternate Data Streams, and encrypted data can evade or slow detection [1]. The agent is user-space (no kernel driver), and hashing large files can cause brief CPU bursts despite metadata gating. In spiking bursts, bounded loss is possible; counters and subsequent verify scans make this clear. Future work: replicate on Linux via the shared `MONITOR` interface, add process/registry telemetry and ADS enumeration to high-risk scopes, and run longer traces to stress false-positive control with a trace sequence.

5.12 Reproducibility

All experiments can be reproduced exactly from artefacts produced during a run. On boot time the agent captures a brief provenance header (config content hash, adapter/agent versions, window size W, UTC), then produces append-only files: `events_*.jsonl` (normalised inputs), `alerts.jsonl` (outputs), `anomaly_state.txt` (rolling n, μ, σ, W), and FIM baselines. Determinism is generated from fixed windowing, pure scoring functions, and canonical path normalisation.

Replay in brief

1. Use the exact configuration whose hash appears in the first line of `alerts.jsonl` and set the same W.
2. Load the matching FIM baseline and `anomaly_state.txt`.
3. Feed the saved `events_*.jsonl` to the pipeline (UTC order).
4. Compare the new `alerts.jsonl` to the original; a byte-for-byte match confirms Explainability (M4) and FR–6/FR–7.

Chapter 6

Evaluation and Discussion

6.1 Chapter purpose

Here, we analyze the evidence that has been produced by the working prototype—from the first *FIM demo* through to the integrated *IntrusionDetectionSML* agent—and reflect on what the results mean for the design. We map the behaviours observed to the goals and constraints defined in Chapter 3 (FR/NFR), comment on the trade-offs that have been made, and describe the limitations and future work. Where possible, we ground the discussion in real artefacts in the repositories (`fim_demo` and `IntrusionDetectionSML`) to maintain a focus on the running system. Beyond pass/fail, we explore *why* each outcome occurred (e.g., how batching, re-stat, and per-folder statistics affected alerts), and what the implications are for operator experience, auditability, and portability.

We project results directly onto the four dimensions of assessment (M1–M4) through the use of JSONL streams, FIM snapshots, and persisted state in order to facilitate deterministic replay and fair comparison across scenarios. We then call out the main risks and residual weaknesses that we observed in testing and explain how the evidence so far confirms the design choices made to date and guides the next iteration.

6.2 What we evaluated and why

The system was assessed along the four axes established above: *detection quality* (M1), *latency* (M2), *footprint* (M3), and *explainability* (M4). These span the operator's experience (is the signal accurate and prompt?), operational realities (is the agent lightweight?), and audit requirements (can each decision be replayed?) [1, 4]. The assessment proceeded in a staged progression that reflects the build path:

1. **FIM-first** (`fim_demo`). We began with a periodic scanner that traversed a sandbox root and produced append-only baselines (path, size, mtime, SHA-256). We checked that controlled edits appeared as *Modified* with hash deltas, creations as *Added*, and deletions as *Removed*, with stable canonical paths (`fim_win.log`. `fim_wrapper_output.log`).

This stage generated ground truth for integrity evidence and checked that immutable baselines + diffs suffice to reconstruct changes unambiguously (supports FR-2/FR-3).

2. **Windows API + recursive monitoring**. Next, the C watcher subscribed to `ReadDirectoryChangesW` and produced normalised lines (time, event, path) for single and nested directories. We stress-tested rename pairing and coalescing under bursty edits, confirming the adapter handled buffer boundaries and produced one compact record per logical change. Outputs matched scripted ground truth in `C:\Test` and then across `C:\TestFolder1` and `C:\TestFolder2`, demonstrating consistent ingestion and path canonicalisation in the face of concurrency (supports FR-1/FR-4, [5]).

3. **Integrated agent** (`IntrusionDetectionSML`). We ultimately enabled per-folder sliding windows, rolling statistics, and a clear fusion rule (thresholds + z-score + optional entropy + allow-list), with JSONL-written alerts and state. Runs across `C:\Windows\System32` and `C:\Users` yielded evidence-rich alerts (`alerts.jsonl`) and precise replay artefacts (`events_C__Users.jsonl`, `anomaly_state.txt`). Aggregation and cool-down reduce duplicate notifications during bursts while preserving counts and hashes for audit, satisfying the design requirements of timeliness, low noise, and reproducibility (supports FR-5/FR-6/FR-7/FR-10).

This sequence keeps variables in check: correctness in a quiet test root (`C:\Test`), then concurrency in `C:\TestFolder1/2`, followed by realism in `C:\Windows\System32` and `C:\Users`, where background services and editor saves provide a representative, mixed workload.

6.3 What the results say about the design

6.3.1 Detection quality (M1)

Throughout all phases the pipeline *consistently* accurately labelled file actions as intended. The normaliser produced a consistent tuple {`timestamp`, `folder`, `path`, `eventType`} and correctly correlated `old`→`new` rename events within a short correlation window, even in light bursts. In the FIM path, *tamper* (content modification) vs *drift* (metadata-only) performed as intended: content edits carried baseline→current hash evidence, while timestamp/size-only modifications were de-escalated. Per-folder windows kept "noisy" user folders from overwhelming, while sensitive paths (e.g., `System32`) had tight thresholds [1]. Ablations verified the design choice to *fuse* absolute thresholds with z-scores: the fusion smoothed-out incidental spikes without muting persistent anomalies, according to anomaly-detection counsel [4]. Standard edge cases (editor temp files, transient renames) were neutralised by the re-stat guard and micro-batching, and therefore did not balloon false positives.

What this achieves. Detector surfaces the *right* events (tamper, long bursts) without false positives from normal churn. Rename semantics and diff evidence make alerts specific and actionable, and the same logic holds from toy origins to `C:\Users` and `C:\Windows\System32`.

6.3.2 Latency (M2)

End-to-end latency from adapter ingestion to JSONL alert was close to the small batching window configured in order not to produce storms. Operations on one file were felt as instantaneous; bursty usage was accumulated and sent within a fixed latency. Infrequent outliers matched expected OS behavior (RDCW coalescing) or an explicit integrity re-hash on changed files [5]. The architecture *intentionally* sacrifices several hundred milliseconds for stability, deduplication, and better operator experience [1].
What this achieves. Alerts in time for triage without generating duplicate floods. Latency patterns are consistent (a prevailing known micro-batch), allowing SLOs and operator expectations to be made easier.

6.3.3 Footprint (M3)

The agent stayed lean at idle and under scripted bursts. Most of the cost came from hashing files with varying metadata; the key SML phases kept per-folder state bounded and streamed append-only lines using minimal in-memory queues. The trusted C surface is small and single-function, and the rest of the pipeline is pure, statically typed SML, which avoids implicit growth and keeps GC effects small.
What this achieves. The deployment meets the "commodity endpoint" goal: low idle CPU, reasonable peaks for bursts, and memory well within the NFR limit. This facilitates constant monitoring without compromising user workloads.

6.3.4 Explainability (M4)

Each alert contains the *minimal but sufficient* evidence to replay the decision: window size W, counts, rolling (μ, σ), computed z, policy hit(s), cool-down state, and—where relevant—baseline→current hashes. Offline replays from the captured configuration content hash duplicated severities byte-for-byte, demonstrating determinism and auditability [1]. Because artefacts are append-only (events, alerts, state, baselines), an independent third party can confirm independently why a decision was taken.
What this achieves. The system is not a black box: each alert is *explainable*, reproducible, and defensible in the case of review. This aids compliance and post-incident investigation directly, without extra instrumentation.

Design takeaways. (i) A lean boundary adapter keeps platform volatility out of sight of detection code; (ii) using simple thresholds with z-scores creates robust signal without complexity; (iii) folding-profiles matter—busy folders learn high baselines while sensitive paths keep narrow bands; and (iv) append-only streams of evidence make testing, tuning, and audits easy.

Axis	Outcome (evidence)
M1 Detection	Correct classification (*tamper/drift*); rename pairing reliable; fused z+thresholds lower noise (alerts.jsonl, fim logs).
M2 Latency	Near micro-batch bound; bounded delay on bursts; outliers tied to RDCW coalescing or re-hash (timestamps).
M3 Footprint	Low idle CPU/mem; peaks dominated by hashing only when metadata changed (process counters).
M4 Explainability	Alerts carry W, counts, (μ, σ), z, policy hits, hash deltas; byte-for-byte replay (events/alerts/state).

Table 6.1: M1–M4 outcome summary.

6.4 How the evidence maps to requirements

The artefacts allow for a concrete trace from observation to requirement. **FR-1/4** occur in live start-up and per-folder counts (`events_*.jsonl`); **FR-2/3** in FIM snapshots and hash-delta alarms (`fim_win.log`, `alerts.jsonl`); **FR-5/6/7** in the fusion severity and persisted rolling state (`anomaly_state.txt`); **FR-8** in deterministic policy evaluation across canonical paths; **FR-9** in the unmodified SML core behind the `MONITOR` signature waiting for `inotify` [6]; **FR-10** in rolled-up incidents with cool-downs. The NFRs of latency, footprint, reliability, testability, and explainability are reflected in the bounded latencies, small memory/CPU, conservative management of overflow, unit-testable pure functions, and reproducible alerts.

FR/NFR	Status (where seen)
FR-1/4	Ingestion/normalisation and per-folder windows (`events_*.jsonl`).
FR-2/3	Immutable baselines and diffs (`fim_win.log`, hash deltas in alerts).
FR-5/6/7	Fusion → JSONL alerts + persisted rolling state (`alerts.jsonl`, `anomaly_state.txt`).
FR-8	Deterministic policy over canonical paths (config hash in alerts).
FR-9	Core unchanged behind `MONITOR` boundary (portability seam).
FR-10	Aggregation, de-dup, cool-downs (burst screenshots, alert grouping).
NFRs	Bounded latency, small footprint, testability, reproducibility (artefacts).

Table 6.2: FR/NFR compliance at a glance.

6.5 Design choices that mattered

Three design decisions made the system both *safe* to run on an endpoint and *effective* at detecting intrusions.

Thin C adapter, strong isolation at the FFI. The Windows watcher is intentionally single-threaded and single-purpose: open a handle, subscribe to `ReadDirectoryChangesW`, and send one normalised line per notification. By not having to contend with complicated buffering, dynamic allocation patterns, or fan-out across many threads, the adapter has a small

crash surface and makes failure modes obvious (bounded buffers, explicit overflow counters). All policy, stateful logic, and parsing are *outside* the SML FFI boundary, so malicious or incorrect input is handled in a memory-safe language. Conservative rename pairing and micro-batching utilize RDCW coalescing without event discarding, and re-stat checks prevent expensive hashing [5]. Overall, this leads to predictable latency and fault resilience without sacrificing kernel-adjacent faults.

Typed normalisation and canonical paths. At entry, every raw line is typed into an SML value of closed action set `(Create|Modify|Delete|Rename)` and canonicalised paths (UTF-16→UTF-8, separator/case rules). Total pattern matching enforces total handlers for every event type; ill-formed inputs are refused early with a reason and no "best-effort" parsing seeps into detection. Canonicalisation also excludes duplicate keys and aliasing (e.g., path casing differences), which is extremely important for integrity diffs and per-folder stats. Overall, the effect is larger *accuracy* (fewer misclassifications) and larger *trust* in every downstream decision (M1).

Immutable evidence: append-only JSONL and sealed baselines. Alerts, rolling statistics, and baselines are written as append-only JSONL with stable field names and a content hash of the config in each record. Baselines are stored as immutable evidence; updates require an explicit approval process rather than "learning" from changes at runtime. As the same pure functions are applied by offline replay and online pipe, each alert can be recomputed byte-for-byte from its evidence tuple (W, μ, σ, z) and (where applicable) baseline→current hashes. This makes the system tamper-evident, audit-ready, and *explainable by construction* (M4), in line with host-IDS guidelines [1].

Per-folder windows and transparent fusion. Highly active user folders legitimately cause more churn than sensitive folders like `System32`. By keeping per-folder sliding windows of rolling (μ, σ), the system learns local baselines and scores the current rate with a guarded (z)-score; absolute thresholds are used per folder class. An open fusion rule (thresholds OR z, escalated by FIM *tamper*, optionally informed by entropy) fuses several weak cues into one clear severity without a black-box model [1, 4]. Cool-downs and de-duplication suppress alert storms under bursts, while re-stat before hashing eliminates editor temporary files and short-term artifacts. The outcome is less false-positive pressure with no reduction in sensitivity to real intrusions (M1/M2).

Why this makes the system safe and effective. The trusted code is minimal and small; all complex logic runs in a strongly typed, memory-safe core. Inputs are canonical and validated, decisions are deterministic and reproducible, and evidence is sealed. Combined these decisions reduce operational risk, increase alert fidelity, and render the intrusion signals both timely and defensible upon investigation.

6.6 Where it struggled and how it was handled

Three practical pain-points were encountered during testing, each based on real OS semantics or workload behavior, and each addressed with targeted defenses.

Rename chains under heavy churn. Under Windows, long rename/move chains can be delivered out of order or coalesced under buffer pressure [5]. In early testing this created spurious delete+create pairs before receipt of the matching *old→new* notification. The pipeline solves this by (i) enforcing a tiny micro-batch window to gather closely spaced notifications, (ii) matching renames in a bounded correlation window, and (iii) processing unmatched pairs conservatively (logged as provisional and reconciled on arrival of the counterpart). Structured diagnostics (overflow counters, queue depth) make such occurrences visible and auditable rather than silent. The trade-off is a slight, bounded latency increase during bursts in exchange for consistent, deduplicated events.

Cost of hashing large files. Full-content hashing is I/O and CPU intensive when large files churn. In order to keep the agent light, integrity verification is *gated* by metadata: if neither `size` nor `mtime` changed, hashing is skipped; if they did, hashing is performed and the baseline→current digest is logged as tamper evidence [8]. This simple guard cut out unnecessary work in daily edits. For directories that are known to contain very large assets, policy can either lower sensitivity (metadata-only drift) or postpone re-hash until the system is quiet; if future deployments require it, chunked/streamed hashing can be added without altering evidence format.

High-churn noise in user caches. browser profiles, build artefacts) can drown out folder statistics, hiding interesting bursts. Rather than hard-coding ignore rules, the design pushes this into the *policy layer*: exclusions are explicit, versioned alongside the configuration content hash, and applied before counting/FIM so as not to introduce bias into (μ, σ). For in-scope but noisy directories, per-directory windows enable baselines to shift upwards whilst sensitive paths (e.g., `System32`) have narrow bands, without over-alerting or decreasing coverage [1]. This also renders noise control an auditable decision, instead of an incidental side-effect..

6.7 Relation to practice and portability

This behavior is in accordance with operational guidelines: integrity baselines for high-value pathways, event-driven inspection, pre-presentation aggregation, and concise, machine-readable evidence [1]. Compared to heavyweight agent–manager stacks, the prototype has a small, auditable core with room for external dashboards. The `MONITOR` interface cleanly decouples OS details; a Linux `inotify` adaptor can satisfy the same signature without modifying detection logic [6]. This allows for comparative research across OSes and future additions without core refactoring.

From an operational standpoint, append-only JSONL with consistent field names fits well into common SIEM ingestion patterns and compliance workflows, and provenance headers (configuration hash, window size, UTC) along with deterministic replay substantially facilitate audit and incident review [1]. On portability, differences in event semantics (e.g., RDCW merging vs. `inotify` edge cases) are within the adapter contract; unit tests at the signature boundary ensure that once an adapter emits the normalised tuple {`timestamp`, `folder`, `path`, `eventType`}, the same SML pipeline makes the same decisions on all platforms [6] extensions.

6.8 Threats to validity and limitations

Results were gathered on a single host with managed workloads; multi-day low-and-slow campaigns and multi-host spread are not considered. Windows `ReadDirectoryChangesW` semantics (coalescing, buffer limits) differ from Linux `inotify`; quantitative comparisons between OS are future work [5, 6]. FIM is blind to purely in-memory tampering; augmenting file evidence with process/registry telemetry would raise coverage [1].

6.9 Comparison of Windows and Linux File–Event Monitoring

This project targets two mature, but semantically different, kernel notification channels: Windows `ReadDirectoryChangesW` (RDCW) and Linux `inotify`. Both provide near–real-time directory change notifications, but they differ in watch granularity, rename semantics, overflow behavior, and path handling. The adapter interface (`MONITOR` signature) used in this project reconciles these differences by yielding a common, typed tuple {`timestamp`, `folder`, `path`, `eventType`}, so the SML detection core (normalisation, counting, FIM, fusion) is the same on both platforms [5, 6].

Scope and watch model

- **Windows (RDCW).** A directory handle can be subscribed to a mask of changes and, with `bWatchSubtree=TRUE`, receive notification recursively for all descendants. This makes whole-tree monitoring straightforward.
- **Linux (`inotify`).** Watches are set up per directory; recursive coverage is implemented by traversing the tree and installing a watch on each subdirectory (and adding/removing watches as directories appear/disappear).

Event types and rename semantics

- **Windows.** Renames are delivered as a pair of events: *old name* followed by *new name*. At load time, coalescing of notifications and slight deferral of pairing are possible.

- **Linux.** Renames are delivered as `IN_MOVED_FROM` (nonzero cookie) and `IN_MOVED_TO` (same cookie). Pairs may straddle directories; both watches are required in order to observe both sides.

In the adapter, both models are normalised to a single logical `RENAME(old→new)` event using a short correlation window (cookie on Linux; near-term pairing on Windows).

Overflow, batching, and back-pressure

- **Windows.** RDCW buffers to a caller-provided buffer. In bursts, events can compound; if buffers are short, entries may get lost. The adapter uses conservative buffers and micro-batch window to prevent duplication and increments the overflow counter when loss can be done.
- **Linux.** `inotify` buffers events into the kernel buffer; overflow causes `IN_Q_OVERFLOW`. The adapter catches this and causes a bounded re-verification scan of affected directories.

Both log structured diagnostics (overflow count, queue depth) and depend on the same de-duplication and cool-down logic downstream.

Path handling and encoding

- **Windows.** Paths are UTF-16; long paths may require the `\\?\` prefix; case is almost case-insensitive on NTFS. The adapter canonicalises to UTF-8, normalises separators, and applies a stable case policy before passing to SML.
- **Linux.** Paths are byte strings (usually UTF-8) and case-sensitive. The adapter passes canonical UTF-8 and normalises `.` / `..` components.

This project's normaliser makes the SML core view canonical, canonical UTF-8 on all platforms to prevent aliasing in the baseline and stats..

Coverage details and edge cases

- **Symbolic links / reparse points.** RDCW can reveal reparse points; directory entries are tracked by `inotify`. Symlinks are treated carefully by the recursive walker to prevent cycles; policy can skip some mount points.
- **Cross-directory moves.** On Windows, both will be visible if hidden by the recursive handle; on Linux, both destination and source directories must be tracked in order to view paired `IN_MOVED_FROM/TO`.
- **Network/mounted file systems.** The behavior will differ on SMB/NFS; the adapter treats these all the same but flags mounts in diagnostics.

Performance considerations

- **Windows.**Recursive monitoring with a single handle makes it more convenient to use resources; coalescing reduces kernel/user transitions but may slow down individual notifications slightly under load.
- **Linux.** Tall trees require many watches (one per directory), so the adapter keeps a map of watches and handles directory create/delete by adding/removing watches.

No variation has any effect on the SML core: event rate is bucketed by folder, and hashing is gated by metadata on both platforms.

Implications for the SML pipeline

The `MONITOR` signature wraps three operations—`start`, `subscribe`, `stop`—and a single event tuple. Platform specifics (rename cookies, recursive flags, buffer size) are confined to the adapter. Post-normalisation, the detection pipeline (normalise → window → score → FIM diff → fuse → alert) is OS-agnostic; same unit tests for parsing, pairing, and replay can be run unchanged.

Aspect	**Windows (`ReadDirectoryChangesW`)**	**Linux (`inotify`)**
Recursive coverage	Single handle with `bWatchSubtree=TRUE`	Per-directory watches; recursion managed in user space
Rename pairing	Old→new pair; may coalesce under load	`IN_MOVED_FROM/TO` with cookie across dirs
Overflow signal	Buffer coalescing/loss (implicit); adapter counter	`IN_Q_OVERFLOW` explicit event
Encoding / case	UTF-16; mostly case-insensitive	UTF-8 bytes; case-sensitive
Watch cost	Low (few handles)	Higher (many watches on large trees)
Adapter task	Buffer sizing, micro-batch, pair old/new	Maintain watch map, match cookies, handle overflows
SML core impact	None (uniform tuple)	None (uniform tuple)

Table 6.3: Practical differences and their containment at the adapter boundary.

Bottom line

RDCW enables straightforward recursive watching with corresponding rename records;`inotify` provides direct rename cookies and direct overflow indications in return for directory-level watches. Encapsulating the disparity in the adapter and emitting a canonical event tuple, the SML detection core and evidence model (JSONL alarms, baseline diffs, rolling state) are identical on both platforms [5, 6].

External validity is limited by hardware and environment : filesystem capabilities, background tasks (AV, backup), and storage medium (SSD vs. HDD) may change latency

and event mix. Bias in measurement can result from the fact that ingress/egress timestamps are taken on the same host; although a monotonic clock reduces skew, OS coalescing might still blur one-to-one correlation in rename storms. Finally, gating re-hash on metadata change saves cost but can miss infrequent edits that preserve both size and mtime; this was not observed in our experiments but is a theoretical corner case.

6.10 Where the Evidence Lives (Repository View)

The discussion references artefacts present in the GitHub directory `https://github.com/shreyas245/Standard-ML/tree/main/Sumeet`. In `IntrusionDetectionSML` the SML core includes `Tailer.sml`, `AnomalyDetection.sml`, `AlertEngine.sml`, `StatePersistence.sml`, and `visualExport.sml`, with adapter traces and large event streams such as `events_C__Users.jsonl`, plus `alerts.jsonl` and `anomaly_state.txt`. In `fim_demo.zip`, the FIM utilities include `fim_monitor.sml`, `fim_realtime.sml`, wrappers (`fim_rdchanges_watch.exe`, `fim_rdchanges_watch.dll`), and logs (`fim_win.log`, `fim_log_2025-08-21.log`), which correspond to the staged FIM experiments.

Inspecting the intrusion-detection artefacts. For the integrated agent, evidence is processed in the following order: (i) the primary output stream `alerts.jsonl` where each line is a complete alert record with `timestamp`, `folder`, `path`, `eventType`, `policyHit`, z-score, severity, and (for changes) `baselineHash`→`newHash`; (ii) `anomaly_state.txt` holds the rolling tuple (W, n, μ, σ) by folder for replay and continuity checks; (iii) the normalised inputs `events_*.jsonl` (e.g., `events_C__Users.jsonl`) supply the ground-truth event timeline used by the tailer and windowing logic; and (iv) any FIM baseline snapshots supply immutable reference rows (path, size, mtime, sha256) to supply integrity diffs. Cross-linking the files rebuilds the full decision path: an event line in `events_*jsonl` is related to a window count and z in `anomaly_state.txt`, which, together with policy and optional hash deltas, generates the matching alert line in `alertsjsonl`.

6.11 Future work

The prototype has a crisp separation of a lean C adapter and a crisp SML core. This enables subsequent steps to verify well: SML's strong static typing, algebraic data types, modules/functors, and immutability provide that fresh capabilities can be added as little, provable enhancements without destabilizing legacy behaviour. Below are strategic additions that build on top of current artefacts and FR/NFRs.

- **Portable Linux adapter (`inotify`).** Integrate a Linux watchdog compatible with the existing `MONITOR` signature and normalises events into the same tuple {`timestamp, folder, path, eventType`}. Because detection core never inspects platform-dependent structures, the change is confined to a single module; cross-OS parity tests can subsequently compare semantics (overflow/coalescing, rename sequences) head-to-head [5, 6].

- **Signed-update workflow and scoped rules for sensitive trees.** Enhance the policy layer with a minimal rule set for precious subtrees (e.g., `System32`). Add a signed "change package" format: approved updates are logged as a diff against the sealed baseline and signed; at run-time, FIM treats those diffs as *authorised* and marks everything else as *tamper*. SML's pattern matching makes the rule evaluation complete and auditable, in accordance with compliance guidelines [1].

- **Incremental hashing for large files.** Use chunked or rolling digests (e.g., block-wise SHA-256 with a Merkle index) so content verification only reads changed regions. The existing FIM interface already separates metadata gating from content hashing; adding an `incremental_sha` implementation preserves the pure-diff contract while improving throughput on large artefacts [8].

- **Thin visual front-end over JSONL.** Build a minimal viewer that renders the ongoing streams (`events_*jsonl`, `alerts.jsonl`) as rate timelines, heatmaps, and drill-down tables. Because the schema is fixed and self-describing (proof included on a per-alert basis), there are no API changes; the UI can remain a stateless reader with export and replay capabilities.

- **Drift modelling and adaptive bands.** Add a slow, bounded "trend" component for active user folders so normal work patterns drift slowly without masking sudden bursts. In SML, this is a pure function layered over the current (μ, σ) estimator, with acceptance tests ensuring detectability of injected spikes is maintained [4].

- **Property-based testing and fuzzing at the FFI boundary.** Employ generators to create adversarial sequences (deep rename chains, lengthy paths, coalesced events) and verify invariants (no crashes, no duplicated alerts per logical action). Referential transparency of SML makes such tests deterministic and reproducible.

- **Tamper-evident logs and baseline provenance.** Chain JSONL records with per-line hashes and a daily root signature so alert streams and baselines are verifiably intact. This change is local to the export module and strengthens audit trails without touching detection logic [1].

- **Multi-source enrichment.** Correlate file events with light process metadata (signer info, creator PID) at the adapter edge. The SML core consumes a richer `eventType` sum type but has the same fusion logic, improving triage without introducing noise.

- **Policy DSL as an SML functor.** Package policy (allow-lists, class limits, cool-downs) as an argument functor to the core; other policies (e.g., more restrictive server profiles) are compile-time options with exhaustive type-checked cases.

Why SML helps. Each of the items above is an intentional design behind an existing signature: the type system prevents accidental schema drift; pure functions and immutability make testing and replay trivial; functors enable implementation switching (Windows/`inotify`,

basic/incremental hashing, strict/relaxed policy) without altering the essential pipeline. These properties reduce integration risk without compromising explainability and auditability that Chapter 3 put forward as first-class goals.

6.12 Summary

The staged development—from an isolated FIM demo all the way up to a monolithic Windows event adapter and finally the combined *IntrusionDetectionSML* agent—resulted in a working FM-IDS that is small, determinate, and auditable. By reducing the C surface and driving normalisation, scoring, and fusion into pure, typed SML modules, the system is robust to bursty workloads while being easy to test and replay. Append-only JSONL artefacts (events, alerts, rolling state, baselines) make every decision reproducible; per-folder stats and an open fusion rule give good signal without murky modelling. In total, the implementation satisfies the Chapter 3 FR/NFRs and has a clean seam (`MONITOR`) for Linux portability.
In one view:

- **Correctness.** FIM *tamper/drift* and event parsing functioned as designed; rename pairing and re-stat guards handled common edge cases.
- **Timeliness.** End-to-end alerting was close to the small batching window; aggregation prevents storms without hiding real spikes.
- **Footprint.** CPU and memory were minimal; hashing overhead is only incurred when metadata indicates a real change.
- **Explainability.** Each alert contains counts, W, (μ, σ), z, policy hits, and hash deltas where relevant; offline replays are byte-for-byte identical.
- **Engineering fit.** Typed normalisation and pure functions simplified testing and tuning; the adapter boundary holds OS differences at arm's length.
- **Ready next steps.** The core is now mature enough to add an `inotify` adapter, scoped rules for sensitive trees, and incremental hashing without changing detection logic.

In short, the prototype performs its marketed purpose: an explainable, lightweight host monitor that translates OS file events into compact, evidence-rich alerts, with a clear path forward for cross-platform testing and increasingly robust controls. .

Chapter 7

Summary

7.1 Overview

The dissertation sought to build a small, auditable file–monitoring intrusion detection system (FM-IDS) that is locally run, explains its judgments, and is easily portable to other operating systems with minimal modification. Progress incrementally went from a standalone File Integrity Monitoring (FIM) demo to a full-fledged agent combining real-time Windows file events with integrity diffs and rudimentary statistical analytics. The heart is written in Standard ML (SML) for readability and safety; a lightweight C bridge links the operating system notifications to the typed SML pipeline. Everywhere, design favored strong typing, immutability, and append-only evidence so that all the alerts can be reconstructed and audited.

The system was exercised from quiet test roots to practical places (`C:\Users`, `C:\Windows\System32`), producing predictable JSONL outputs that make decisions traceable and simple to audit. A consistent `MONITOR` interface cleanly separates OS details, so the same SML core may be used with a future Linux `inotify` adapter without changing the detection logic.

7.2 Summary of Key Findings

- **Safe split.** A lightweight C observer (`ReadDirectoryChangesW`) operating a typed SML core kept parsing/policy/state in SML, reducing crash surface and making behaviour deterministic.
- **Windows semantics handled.**Coalesced events and rename pairs old→new were stabilised by a short micro-batch and conservative pairing; UTF16→UTF 8 path canonicalisation maintained stable keys.
- **Clear integrity signals.** Immutable FIM baselines with re-stat gating (hash only when metadata changes) cleanly separated *tamper* from *drift*.
- **Useful behavioural layer.** Per-folder windows with (μ, σ) supplemented by a transparent fusion (thresholds OR z, boosted by FIM) produced comprehensible severities

while cool-downs prevented storms.

- **Reproducible alerts.** Bounded JSONL (scope, policy hit, z, hashes, config hash) made every decision replayable and audit-ready.
- **Lightweight and portable.** Bounded state and append-only I/O kept CPU/memory low; the `MONITOR` signature keeps OS details separate, with the SML core unchanged for future `inotify` adaptation.

7.3 What was built

The project yielded a working FM-IDS with the following concrete outcomes:

- **Code.** A trim Windows monitor (C) and SML modules for tailing, anomaly statistics, policy and fusion, FIM comparison, and state persistence.
- **Evidence streams.** `events_*.jsonl` (normalized inputs), `alerts.jsonl` (evidence and severity decisions), `anomaly_state.txt` (rolling W, n, μ, σ), and FIM baselines (append-only snapshots).
- **Operator-ready schema.** An uniform alert format (timestamp, scope, event type, policy hit, z-score, integrity deltas, severity) ready for dashboards and audit pipelines.

7.4 How it addresses the requirements

The system meets the functional and non-functional requirements set up earlier:

- **FR (capabilities).** Realtime ingestion, immutable baselines and diffs, per-folder rolling statistics, fused severity, JSONL export, persisted state, configurable policy, adapter abstraction, and burst-resilient aggregation.
- **NFR (qualities).** Messages arrive in timely and bounded fashion; memory and CPU are low by design (metadata gating before hashing, bounded state); all choices are deterministically reproducible from preserved evidence; configuration is deterministically checked; and the core becomes portable behind a stable signature.

7.5 Closing statement

This paper offers an operational file-monitoring intrusion detection system with an explicit architecture, a well-founded evidence model, and a real-world path to portability. The simulated progression from FIM to Windows API use to an SML pipeline resulted in a tool that is correct for what it reports, light in how it works, and understandable in why it warns. The code and artefacts provide a solid foundation to iterate further with the underlying purposes of clarity, safety, and explainability sustained.

Above all else, a light-weight, well-isolated C adapter and statically typed SML core give deterministic behavior and reproducible alarms, and append-only evidence (events, state, baselines) makes all decisions end-to-end auditable. The stable `MONITOR` boundary ensures OS-specific data are encapsulated, and the standard JSONL schema maps naturally to operator workflows and compliance requirements. As a whole, the system converts raw OS file events to condensed evidence-rich records with expected performance and verifiable origin, satisfying the given FR/NFRs and displaying a uniform, production-oriented design.

Bibliography

[1] K. Scarfone and P. Mell, "Guide to intrusion detection and prevention systems (idps)," NIST, Tech. Rep. Special Publication 800-94, 2007.

[2] M. Roesch, "Snort—lightweight intrusion detection for networks," in *Proceedings of the USENIX LISA Conference*, 1999.

[3] A. Patcha and J.-M. Park, "An overview of anomaly detection techniques: Existing solutions and latest technological trends," *Computer Networks*, vol. 51, no. 12, pp. 3448–3470, 2007.

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[5] Microsoft, "Readdirectorychangesw function," https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-readdirectorychangesw, 2023.

[6] R. Love, *Linux Kernel Development*, 3rd ed. Addison-Wesley, 2010.

[7] G. H. Kim and E. H. Spafford, "The design and implementation of tripwire: A file system integrity checker," in *Proceedings of the 2nd ACM Conference on Computer and Communications Security (CCS)*, 1994, pp. 18–29.

[8] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.

[9] JSON Lines, "Json lines — the jsonl format," https://jsonlines.org/, accessed: 2025-09-12.

[10] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987.

[11] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013.

[12] O. Cervantes, D. Ortiz, and R. Rodriguez, "Ossec: An open source solution for host-based intrusion detection," in *Latin American Network Operations and Management Symposium (LANOMS)*, 2008.

[13] OSSEC Project, "Ossec documentation," https://www.ossec.net/, 2022.

[14] Wazuh Project, "Wazuh documentation," https://documentation.wazuh.com/, 2023.

[15] PCI Security Standards Council, "Payment card industry data security standard (pci dss) v3.2.1," https://www.pcisecuritystandards.org/, 2018.

[16] R. Bird, *Thinking Functionally with Haskell*. Cambridge University Press, 2014.

[17] L. C. Paulson, "Mechanized proofs for a recursive authentication protocol," in *Proceedings of the 10th IEEE Computer Security Foundations Workshop (CSFW)*, 1997.

[18] F. Pottier and D. Rémy, "The essence of ML type inference," in *Advanced Topics in Types and Programming Languages*, B. C. Pierce, Ed. MIT Press, 2005, pp. 389–489.

[19] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[20] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Addison-Wesley, 2012.

Appendices

Appendix A

Operational Appendix (Build, Run, and Artefacts)

A.1 Appendix E — List of Abbreviations

Abbreviation	Meaning / Usage in this dissertation
FM-IDS	File–Monitoring Intrusion Detection System (the prototype built in this work).
IDS	Intrusion Detection System.
HIDS	Host–Based Intrusion Detection System.
FIM	File Integrity Monitoring (baseline, diff, tamper/drift).
FFI	Foreign Function Interface (SML ↔ C boundary).
SML	Standard ML (core implementation language).
API	Application Programming Interface (e.g., Windows file event API).
OS	Operating System (Windows, Linux).
RDCW	`ReadDirectoryChangesW` (Windows directory change notification API).
`inotify`	Linux file–event notification facility.
JSON	JavaScript Object Notation (structured records).
JSONL	JSON Lines (line–delimited JSON; one JSON object per line for append–only logs).
CSV	Comma–Separated Values (flat file export for events).
FR	Functional Requirement (Chapter 3).
NFR	Non–Functional Requirement (Chapter 3).
SIEM	Security Information and Event Management (downstream consumer of alerts).
UTC	Coordinated Universal Time (timestamps in logs).
UTF–8 / UTF–16	Unicode Transformation Formats (path/text encodings; adapter normalises to UTF–8).
I/O	Input/Output (file, console, pipe operations).
CPU	Central Processing Unit (utilisation metric in footprint tests).

Abbreviation	Meaning / Usage in this dissertation
MB	Megabytes (memory metric in footprint tests).
SHA–256	Secure Hash Algorithm, 256–bit (content digest for FIM evidence).
GC	Garbage Collection (runtime memory management; mentioned for footprint).
ADT	Algebraic Data Type (typed event/policy representations in SML).
DSL	Domain–Specific Language (policy configuration concept).
EDR	Endpoint Detection and Response (scope distinction vs. this prototype).
AV	Anti–Virus (background service impacting workloads).
P95	95th percentile (latency reporting).
IQR	Interquartile Range (latency dispersion metric).
SSD / HDD	Solid–State Drive / Hard–Disk Drive (storage type affecting timings).
UCRT64	Universal C Runtime (MSYS2 toolchain used to build the Windows adapter).
MSYS2	Minimal SYStem 2 (Windows development environment used for builds).

A.2 Appendix B — `IntrusionDetectionSML`

Purpose

An SML file monitor agent that takes Windows file-events through a thin adapter, performs per-folder sliding-window analytics (counts, μ, σ, z), enforces policy, and outputs compressed JSONL records for dashboards and replay

Verified folder layout (from the archive)

```
IntrusionDetectionSML/
 IntrusionDetectionSML/
    eventTypes.sml
    dataLogger.sml
    policyRules.sml
    anomalyDetection.sml
    mlAnomaly.sml
    hashing.sml
    alertEngine.sml
    visualExport.sml
    osDetect.sml
    statePersistence.sml
    fimSnapshot.sml
    monitorLauncher.sml
    tailer.sml
    intrusionMain.sml
    watcher.exe            # Windows adapter (provided)
    winWatcher.exe         # Alternate Windows adapter (provided)
    anomaly_state.txt      # Rolling stats state (persisted)
    events.jsonl           # Normalised event stream
    events_C__Users.jsonl  # Large sample event stream
    visual_log.jsonl       # JSONL with level/evidence
    watcher_output.txt     # Adapter console capture
    events.csv             # CSV export

```

Notes. The adapter binary is included already (watcher.exe/winWatcher.exe). No Linux adapter is shipped with this package; portability is attained through the shared MONITOR signature and can be delivered through the use of inotify in follow-on work.

Run-time prerequisites

- Windows 10/11 x64.
- Poly/ML (or another SML system capable of `use` and standard basis).
- A terminal with the working directory set to `IntrusionDetectionSML`.

Start-up sequence (Windows)

1) Load the SML modules in order.

```
use "eventTypes.sml";
use "dataLogger.sml";
use "policyRules.sml";
use "anomalyDetection.sml";
use "mlAnomaly.sml";
use "hashing.sml";
use "alertEngine.sml";
use "visualExport.sml";
use "osDetect.sml";
use "statePersistence.sml";
use "fimSnapshot.sml";
use "monitorLauncher.sml";
use "tailer.sml";
use "intrusionMain.sml";
```

What this does. It builds the typed pipeline: event types → counting → anomaly stats → fusion → exports. State persistence is loaded so the agent can recover (W, n, μ, σ) from `anomaly_state.txt`. No files are written yet.

2) Launch the agent on one or more roots.

```
(* Single root *)
IntrusionMain.main ["C:\\Test"];

(* Multiple roots *)
IntrusionMain.main ["C:\\TestFolder1", "C:\\TestFolder2"];

(* Realistic paths *)
IntrusionMain.main ["C:\\Users", "C:\\Windows\\System32"];
```

What this does. The SML core generates `watcher.exe`, begins tailing line-delimited stream, normalises each record into typed event, updates per-folder windows, computes z-scores, enforces policy, and outputs JSONL/CSV artefacts. Rolling state is appended to `anomaly_state.txt` at regular intervals and at exit.

Produced artefacts (names as written by the code)

- `events.jsonl`, `events_C__Users.jsonl`: normalised events (timestamp, folder, path, event).
- `visual_log.jsonl`: enriched JSON lines with decision `level` and minimal evidence for replay.
- `events.csv`: convenience export of normalised events.
- `anomaly_state.txt`: per-folder rolling tuple (W, n, μ, σ) for persistence and replays.

Example outputs (taken from the archive)

Normalised event (JSONL).

```
{"time":"2025-08-28 08:53:13","event":"RENAME",
 "path":"C:\\Users\\Example\\rename.tmp","folder":"C:\\Users"}
```

Explanation. One logical rename with canonicalised path; this is the tailer's input to counting and statistics.

Visual export with level (JSONL).

```
{"time":"2025-08-28 08:46:50","event":"MODIFY",
 "path":"C:\\Users\\renamed.txt_2025-08-28 08:46:50",
 "folder":"C:\\Users","level":"WARN"}
```

Explanation. The `visualExport.sml` module emits a compact alert-like line with severity `level` from merged scoring. Evidence for replay (counts, W, μ, σ) is stored in `anomaly_state.txt`.

Module responsibilities (quick map)

```
eventTypes.sml         # Typed actions and event records
tailer.sml             # Line -> typed event; per-folder bucketing
anomalyDetection.sml   # Rolling (n, mu, sigma), z-score
mlAnomaly.sml          # Entropy and helper analytics
policyRules.sml        # Allow-list, folder classes, limits
alertEngine.sml        # Fusion: thresholds + z(+entropy) -> level
visualExport.sml       # JSONL export of decisions/evidence
statePersistence.sml   # Append-only (anomaly_state.txt)
monitorLauncher.sml    # Spawn/pipe adapter (watcher.exe)
hashing.sml/fimSnapshot.sml
                       # Baseline scanning utilities
```

A.3 Appendix C — `fim_demo`

Purpose

A standalone File Integrity Monitoring (FIM) demo in SML that takes a root, takes a baseline (path, size, mtime, hash), and reports *Added/Removed/Modified* on a periodic scan interval. Windows leverages the provided watcher tools for real-time experimentation.

Verified folder layout (from the archive)

```
fim_demo/
 fim_monitor.sml
 fim_realtime.sml
 fim_win.log                # FIM log (sample)
 fim_log_2025-8-21.log      # FIM scan log (sample)
 fim_rdchanges_watch.exe    # Windows watcher (provided)
 fim_rdchanges_watch.dll    # Support library (provided)
 fim_win_watcher.exe        # Alternate watcher (provided)
 fim_win_wrapper.c          # C wrapper (source)
 c_changes_wrapper.exe      # Helper (sample)

```

Run (periodic scan)

```
(* In Poly/ML within the fim_demo directory *)
PolyML.use "fim_monitor.sml";

(* Monitor a folder every 5 seconds *)
FIM.main ["C:\\Test", "5"];
```

What this does. Creates a baseline of `C:\Test` and subsequently rescans 5-second intervals, outputting *Added/Removed/Modified* and (on modify) the hash delta proof to `fim_win.log`.

Run (Windows real-time experiments)

```
(* In Poly/ML within fim_demo *)
PolyML.use "fim_realtime.sml";

(* Use provided watcher; write events to fim_events.log *)
FIMRealtime.main ["fim_rdchanges_watch.exe",
                  "fim_events.log",
                  "C:\\TestFolder1",
                  "C:\\TestFolder2"];
```

What this does. Executes the native Windows observer and streams events from the indicated folders into `fim_events.log`, which you can contrast with FIM diffs.

Example log snippets (from the archive)

```
[2025-08-23 01:44:49] C:/TestFolder1 | New folder created: C:/TestFolder1/TestNewFolder
[2025-08-23 01:44:52] ALERT: Too many changes detected in C:/TestFolder1
```

A.4 Appendix D — Data formats (JSONL/CSV schemas)

Normalised event JSONL (`events*.jsonl`)

```
{"time":"YYYY-MM-DD hh:mm:ss",
 "event":"CREATE|MODIFY|DELETE|RENAME",
 "path":"<full canonical path>",
 "folder":"<root folder>"}
```

Semantics. One record per logical change emitted by the Windows adapter and normalised by the tailer.

Decision JSONL (`visual_log.jsonl`)

```
{"time":"YYYY-MM-DD hh:mm:ss",
 "event":"<action>",
 "path":"<path>", "folder":"<root>",
 "level":"INFO|WARN|CRITICAL"}
```

Semantics. `level` is computed from the interaction of thresholds, z, and policy. The numerical evidence upon which it is derived (counts, W, μ, σ) is preserved in `anomaly_state.txt`.

CSV export (`events.csv`)

```
time,event,path,folder
2025-08-28 08:53:13,RENAME,C:\Users\Example\rename.tmp,C:\Users
```

A.5 Appendix E — Minimal code excerpts

Tailer: normalise and bucket

```
(* tailer.sml: convert adapter lines to typed events and count per-folder *)
structure Tailer =
struct
  datatype action = Create | Modify | Delete | Rename
  type event = { time:int, folder:string, path:string, act:action }

  (* CSV-like: time,folder,path,action -> typed event *)
  fun parseLine s =
    case String.tokens (fn c => c = #",") s of
      (ts::fd::p::a::_) =>
        let
          val act = case String.map Char.toLower a of
                      "create" => Create | "modify" => Modify
                    | "delete" => Delete | "rename" => Rename
                    | _ => Modify
          val t = valOf (Int.fromString ts)
        in SOME { time = t, folder = fd, path = p, act = act } end
    | _ => NONE

  (* Per-folder fixed windows for rate x_t *)
  val winSec = ref 60
  fun bucketKey t = t div (!winSec)
  (* counts[...] holds (bucket -> count) per folder *)
  (* ... *)
end
```

Tailer consumes one comma-separated line per OS notification and parses it into a *typed* SML record `event` with an action set closed over `(Create, Modify, Delete, Rename)`. The parsing is strict on field order ⟨`time`, `folder`, `path`, `action`⟩: invalid lines yield `NONE` (discarded), and unknown actions are conservatively mapped to `Modify` so the pipeline is total. The integer `time` (epoch seconds) is deterministically mapped into a fixed window by `bucketKey` $t = t \,/\, $ `winSec`, partitioning the timeline into non-overlapping windows (default 60 s). In every `folder`, the tailer increments the current bucket in a table-of-tables map (`folder` $\mapsto$ (`bucket` $\mapsto$ `count`)). The resulting per-folder sequence x_t (events per window) is then passed on to rolling statistics (μ, σ) and the z-score during the anomaly phase, to supply downstream logic with robust, rate-based evidence rather than raw bursty lines.

Anomaly: rolling μ, σ and z

```
(* anomalyDetection.sml: Welford updates and a guarded z-score *)
structure AnomalyDetection =
struct
  type stats = { n:int, mean:real, m2:real }
  fun update ({n,mean,m2}, x) =
    let val n' = n + 1
        val d  = x - mean
        val m' = mean + d / Real.fromInt n'
    in { n = n', mean = m', m2 = m2 + d * (x - m') } end

  fun stdev {n,mean=_,m2} =
    if n < 2 then 0.0 else Math.sqrt (m2 / Real.fromInt (n - 1))

  fun zScore (x, s:stats) =
    let val sdev = stdev s
    in (x - #mean s) / (if sdev < 1.0e~6 then 1.0e~6 else sdev) end
end
```

Explanation. A numerically stable update and a small-variance guard prevent spurious spikes, making the fused severity predictable.

Visual export: decision line

```
(* visualExport.sml: JSONL with minimal decision evidence *)
structure VisualExport =
struct
  fun write (time, event, path, folder, level) =
    let
      val line =
        "{\"time\":\"" ^ time ^ "\","
      ^ "\"event\":\"" ^ event ^ "\","
      ^ "\"path\":\""  ^ path  ^ "\","
      ^ "\"folder\":\""^ folder^ "\","
      ^ "\"level\":\"" ^ level ^ "\"}"
    in
      DataLogger.append "visual_log.jsonl" line
    end
end
```

Explanation. Writes one compact record per decision. Full numeric context is available in `anomaly_state.txt` for deterministic replays.

A.6 Appendix F — Reproducibility (byte-for-byte replays)

1. Use the configuration and window size W recorded at the start of the run.
2. Load `anomaly_state.txt` for the same folders.
3. Feed the saved `events*.jsonl` into the same normaliser and windowing.
4. Recompute z and the fusion rule; compare the new JSONL and levels to originals.

Since pure transformations and append-only files are used in the pipeline, the choices are reproducible from artefacts that are part of the archive.

A.7 Appendix H — Troubleshooting (Windows watcher)

- **No event lines.** Make sure `watcher.exe` is in the working directory and has rights to open the watched folders.
- **Rename storms look like delete+create.** This can occur under coalescing; micro batch window in the tailer groups them within a short correlation window.
- **Large-file hashing is slow.** Agent gates hashing by metadata; do not observe high-churn large binary caches, or adjust policy to consider metadata-only drift as lower severity.